# Machine Learning — UAQ

## Presentación

Angel Gutiérrez

25 de marzo de 2026

# Contenido


1. Arranque y framing
2. Teoría: workflow de ML
3. Lab NumPy: baseline + split + métricas
4. Lab sklearn: Pipeline + CV + PR-AUC
5. Cierre y entregable

# Semana 1 — Introducción + workflow de Machine Learning

## Framing, generalización, overfitting, leakage, baselines y métricas (Accuracy + PR/PR-AUC)

Curso de Machine Learning — UAQ (Matemáticas Aplicadas)

Libro base: *Probabilistic Machine Learning: An Introduction* (Kevin P. Murphy)

Semana 1

# Temario de la Semana 1 (qué cubrimos hoy)

1. **Arranque y framing:** dataset $\rightarrow$ modelo $\rightarrow$ predicción $\rightarrow$ decisión
2. **Teoría (slides):**
   - Problem framing (qué es $X$, qué es $y$, unidad de predicción)
   - Generalización y overfitting
   - Leakage (conceptual + ejemplos)
   - Baselines
   - Métricas: Accuracy + Precision/Recall + PR-AUC
3. **Lab NumPy:** split + baseline + accuracy + PR básico (+ mini-leakage)
4. **Lab sklearn:** Pipeline + cross_val_score + PR-AUC
5. **Entregable:** repo + notebook plantilla + checklist reproducible

# Lecturas sugeridas (PML = libro base) — Semana 1

## Lectura previa (fundamentos para la sesión)

- **Cap. 1 — Introduction**
  - ¿Qué es ML? (definición tipo T/P/E) y *supervised learning*
  - *Generalization* y *overfitting* (por qué separar train/test)

## Lectura mínima para métricas (lo que usaremos hoy)

- **Cap. 5 — Decision theory**
  - *Precision*, *Recall*, *Precision–Recall curve*
  - *Average Precision* (PR-AUC) como resumen

## Lectura posterior (para consolidar evaluación honesta)

- **Cap. 4 — Statistics**
  - *Validation set* y selección de hiperparámetros
  - *Cross-validation* (K-fold): por qué reduce varianza con $N$ pequeño

# Semana 1 — mapa del temario (progreso)

1. Arranque y framing
2. Teoría: workflow de ML
3. Lab NumPy: baseline + split + métricas
4. Lab sklearn: Pipeline + CV + PR-AUC
5. Cierre y entregable

# ¿Qué es "aprender" en ML? (visión de extremo a extremo)

- **Dataset** (observaciones): pares $(x_i, y_i)$
- **Modelo** $f_\theta$: transforma $x$ en una predicción $\hat{y}$
- **Métrica**: mide qué tan bien predice (Acc, AP/PR-AUC)
- **Decisión**: usar $\hat{y}$ o un score para actuar (aprobar, clasificar, alertar, etc.)

**Idea central del curso**

El objetivo real no es "ganar en train"; es **generalizar** con evaluación honesta y reproducible.

# Semana 1 — mapa del temario (progreso)

1. Arranque y framing
2. Teoría: workflow de ML
   - Problem framing
   - Generalización y overfitting
   - Fuga de información (leakage)
   - Modelos de referencia
   - Métricas
3. Lab NumPy: baseline + split + métricas
4. Lab sklearn: Pipeline + CV + PR-AUC
5. Cierre y entregable

# Problem framing: qué debes definir antes de entrenar

- **Unidad de predicción:** ¿qué entidad scoreas? (persona, cliente, imagen, transacción)
- **Features $X$:** información disponible **al momento de predecir**
- **Target $y$:** definición operacional (evento + ventana + etiqueta)
- **Salida del modelo:** clase (0/1), probabilidad, score continuo
- **Métrica:** qué significa "éxito" (accuracy, AP, etc.)

**Pregunta guía**

**¿Qué decisión cambia si tengo este score/predicción?**

# Supervised vs unsupervised vs RL (solo para ubicar)

**Supervised**

- Tienes etiquetas $y$
- Objetivo: predecir $y$ desde $x$
- Ej: spam, precio de casa

**Reinforcement Learning**

- Aprendes por recompensa acumulada
- Interacción agente–entorno
- Ej: control, juegos

**Unsupervised**

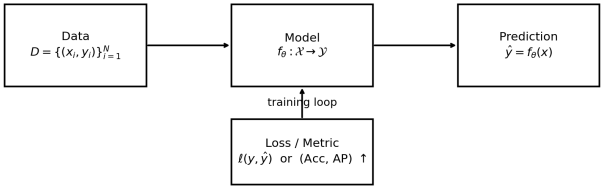
- No hay $y$
- Objetivo: estructura/representaciones
- Ej: clustering, PCA

**Hoy**

Nos enfocamos en **supervised** (clasificación binaria) + evaluación.

# Supervised learning: esquema visual (para fijar ideas)

Supervised learning: learn $f(x)$ from labeled pairs $(x, y)$




- Entrenamiento: ajustar $\theta$ para mejorar una pérdida/métrica en train.
- Evaluación: medir en datos **no usados** para ajustar el modelo.

# Generalización: el objetivo real

- Queremos buen desempeño en datos nuevos (out-of-sample).
- Por eso separamos datos: train / validation / test.

**Intuición**

**Train performance** es "qué tan bien memoriza".
**Test performance** es "qué tan bien generaliza".

# Overfitting: definición + señal práctica

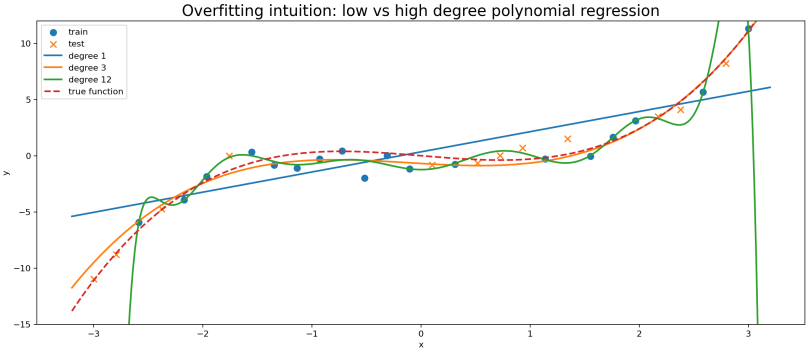
**Overfitting** = el modelo se ajusta demasiado a peculiaridades/ruido del train:

- train muy bien, test/val notablemente peor
- "se derrumba" al salir del dataset de entrenamiento

**Generalization gap**

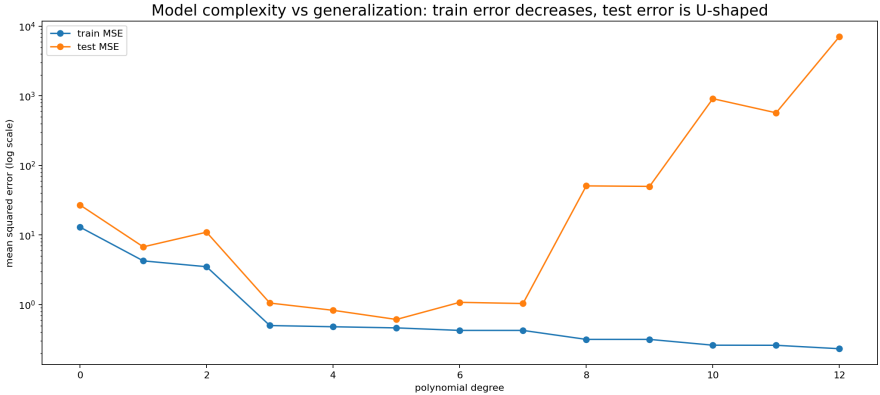
Diferencia entre el desempeño en train y en val/test.

# Overfitting visual (intuición con modelos de distinta complejidad)




- Modelo simple: puede sub-ajustar (*underfit*)
- Modelo muy flexible: puede memorizar train y fallar en test

# Complejidad vs error (por qué el test error es “U-shaped”)




- El **train error** suele bajar con complejidad.
- El **test error** suele tener un mínimo: demasiada complejidad vuelve a empeorar.

# Fuga de información: por qué las métricas pueden mentir

**Pregunta guía**

¿Por qué mi métrica (accuracy / AP) es alta en el cuaderno y cae en producción?

- **Objetivo**: estimar desempeño fuera de muestra (generalización).
- **Problema**: si el proceso "ve" datos del futuro, la evaluación es optimista.
- **Consecuencia**: eliges mal el modelo / umbral $\Rightarrow$ mal despliegue.

# Definición operativa (en el flujo train/val/test)

- Partición: $\mathcal{D} = \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{val}} \cup \mathcal{D}_{\text{test}}$ (disjuntos).
- En el pipeline "se aprenden":

$$\theta \text{ (modelo)}, \qquad \phi \text{ (transformaciones)}, \qquad \lambda \text{ (regularización)}, \qquad t \text{ (umbral)}.$$

## Fuga de información

Ocurre si $\mathcal{D}_{\text{val/test}}$ influye (directa o indirectamente) en $\theta, \phi, \lambda, t$.

## Regla de oro

**Todo ajuste (`fit`) se hace solo con $\mathcal{D}_{\text{train}}$ (o dentro de cada pliegue).**

# Idea clave 1: cuándo el test estima el riesgo (sin sesgo)

- Riesgo (pérdida $\ell$): $R(f) = \mathbb{E}[\ell(y, f(x))]$.
- Estimador en test:

$$\widehat{R}_{\text{test}}(f) = \frac{1}{n_{\text{test}}} \sum_{(x,y)\in\mathcal{D}_{\text{test}}} \ell(y, f(x)) \,.$$

## Condición crítica

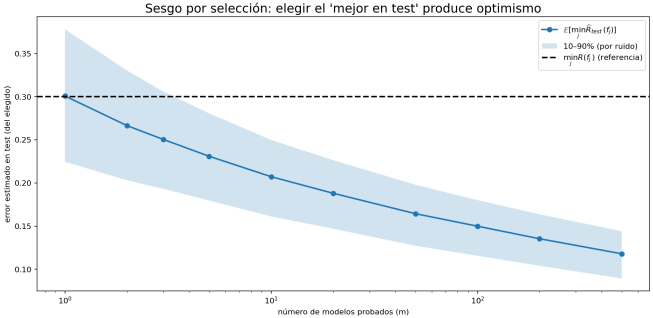
$\widehat{R}_{\text{test}}(f)$ es una evaluación honesta si $f$ es **independiente** de $\mathcal{D}_{\text{test}}$.

- Si el test se usa para decidir $f$ (modelo/umbral/variables), se rompe la independencia.
- $\Rightarrow$ la métrica en test se vuelve **optimista** (sesgo).

# Idea clave 2: sesgo por selección (mirar test para elegir)

- Si usas $\mathcal{D}_{\text{test}}$ para elegir entre $m$ opciones, el "ganador" suele ganar por **suerte**.
- Test finito: $\widehat{R}_{\text{test}}(f_j) = R(f_j) + \varepsilon_j$ (ruido de muestreo).
- Elegir por el mínimo introduce optimismo:

$$\widehat{R}_{\text{test}}(f_{j^\star}) = \min_j \{R(f_j) + \varepsilon_j\} \quad \text{(tiende a ser menor que } \min_j R(f_j)\text{)}.$$



# Idea clave 2 (continuación): cómo evitar el sesgo por selección

## Regla práctica

**El test es para auditar, no para decidir.**

- Decide con $\mathcal{D}_{\text{val}}$ o validación cruzada:

$$\text{modelo},\ \lambda,\ \text{variables},\ t\ (\text{umbral}).$$

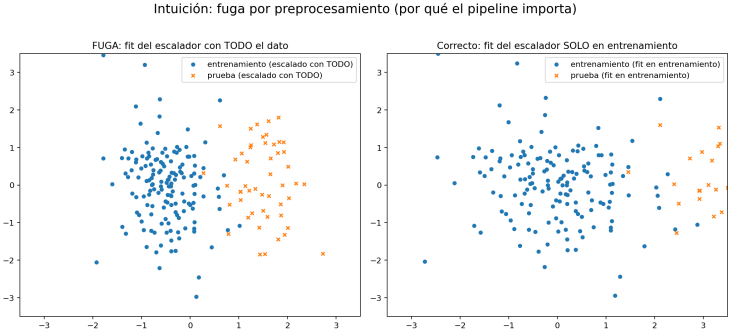
- Usa $\mathcal{D}_{\text{test}}$ **una sola vez** al final (sin "volver a mover perillas").
- Si ajustaste algo tras ver test, entonces ese test ya no es honesto $\Rightarrow$ necesitas un **nuevo test**.

# Idea clave 3: fuga por preprocesamiento (por qué el pipeline importa)

- Transformación: $z = T_\phi(x)$. **Primero** se ajusta $\phi$ en $\mathcal{D}_{\text{train}}$.

$$\phi \leftarrow \mathrm{fit}(T, \mathcal{D}_{\text{train}}) \qquad \Rightarrow \qquad \theta \leftarrow \arg\min_\theta \widehat{R}_{\text{train}}(\theta \mid \phi).$$

- Fuga: $\phi \leftarrow \mathrm{fit}(T, \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{test}})$.



# Ejemplo numérico mínimo: “elegí el mejor en test”

## Mismo mundo, test finito

Supón que (en realidad) $R(f_A) = R(f_B) = R(f_C) = 0{,}30$.

$$\widehat{R}_{\text{test}}(f_A) = 0{,}28, \quad \widehat{R}_{\text{test}}(f_B) = 0{,}24, \quad \widehat{R}_{\text{test}}(f_C) = 0{,}27$$

- Si eliges y reportas $f_B$ por tener 0.24, ya usaste el test para decidir.
- En un test nuevo, el rendimiento típico “regresa” cerca de 0.30.
- Solución: decide con $\mathcal{D}_{\text{val}}$ (o validación cruzada) y **audita** con $\mathcal{D}_{\text{test}}$.

# Checklist: 3 patrones típicos de fuga de información

1. **Preprocesamiento**: ajustar transformaciones con todo el dataset $\Rightarrow$ $\phi$ debe salir solo de $\mathcal{D}_{\text{train}}$ (o por pliegue).
2. **Variable objetivo**: variables que "usan el futuro" o derivadas del desenlace $\Rightarrow$ la variable debe existir al momento de la decisión.
3. **Partición**: duplicados / mismo cliente-paciente en train y test / partición temporal incorrecta $\Rightarrow$ split por grupo o por tiempo cuando corresponda.

**Señal de alarma:** métrica "demasiado buena" + caída fuerte al desplegar.

# Resumen: reglas prácticas (para no auto-engañarnos)

- **Test** se usa para **auditar** (una vez), no para elegir.
- **Validación / validación cruzada** se usa para decidir $\lambda$, variables y umbral $t$.
- Todo `fit` (modelo y preprocesamiento) vive dentro de train (o dentro del pliegue).
- La partición respeta estructura del dato: tiempo, grupos, duplicados.

**Takeaway**

Una métrica solo es creíble si el experimento imita el uso real del modelo.

# Modelo de referencia: definición + por qué es obligatorio

**Modelo de referencia** = modelo simple que establece un "piso" de desempeño.

- **Referencia ingenua:** clase mayoritaria (predictor constante)
- **Referencia fuerte:** modelo lineal simple (regresión logística)

**Disciplina**

No celebramos un modelo si no supera una referencia razonable.

# Métricas (clasificación binaria): matriz de confusión

| | $\hat{y} = 1$ | $\hat{y} = 0$ |
|---|---|---|
| $y = 1$ | $TP$ | $FN$ |
| $y = 0$ | $FP$ | $TN$ |

- $TP$: positivos detectados $FN$: positivos perdidos
- $FP$: falsas alarmas $TN$: negativos correctos

**Idea**

Todas las métricas de clasificación se construyen con $TP, FP, FN, TN$.

# Precisión / Cobertura (recall): definiciones e intuición

- **Precisión** (pureza):

$$\mathrm{Prec} = \frac{TP}{TP + FP}$$

- **Cobertura** (recall):

$$\mathrm{Rec} = \frac{TP}{TP + FN}$$

**Compromiso (umbral)**

Al mover el umbral $t$, cambian $TP, FP, FN \Rightarrow$ cambia (precisión, cobertura).

# Ejemplo mínimo: misma exactitud, comportamiento distinto

## Dataset desbalanceado

$N = 100$: 10 positivos, 90 negativos.

## Modelo A (ingenuo): $\hat{y} = 0$ siempre

$$TP = 0,\ FN = 10,\ FP = 0,\ TN = 90$$

$$\mathrm{Acc} = 0{,}90, \qquad \mathrm{Rec} = 0$$

## Modelo B: detecta algunos positivos

$$TP = 6,\ FN = 4,\ FP = 6,\ TN = 84$$

$$\mathrm{Acc} = 0{,}90, \quad \mathrm{Prec} = 0{,}50, \quad \mathrm{Rec} = 0{,}60$$

## Mensaje

La **exactitud** puede ocultar fallas en la clase positiva $\Rightarrow$ usamos precisión/recall y curva PR.

# Exactitud (accuracy): cuándo sirve y cuándo engaña

**Exactitud (accuracy)**:

$$\mathrm{Acc} = \frac{TP + TN}{TP + TN + FP + FN}.$$

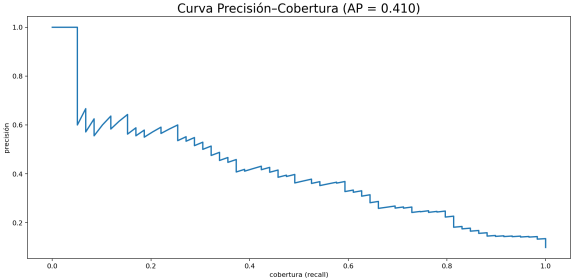
- Útil si las clases están balanceadas y los costos de error son similares.
- Puede engañar con desbalance: puedes tener alta exactitud sin detectar positivos.

**Ejemplo mental**

Si 5 % son positivos, predecir siempre 0 da 95 % exactitud (pero 0 recall).

# Curva Precisión–Recall y AP

- Curva PR: puntos $(\mathrm{Recall}(t), \mathrm{Prec}(t))$ al variar el umbral $t$.
- Se usa cuando hay **desbalance** y nos importa la **clase positiva**.
- Línea base: un clasificador no-informativo tiene $\mathrm{Prec} \approx \pi$, con $\pi = P(y = 1)$.
- **AP** (Average Precision): resumen escalar tipo "área" bajo la curva PR.




## Uso práctico

Elegir $t$ según una meta: recall mínimo o precisión mínima.

# Semana 1 — mapa del temario (progreso)

1. Arranque y framing
2. Teoría: workflow de ML
3. Lab NumPy: baseline + split + métricas
4. Lab sklearn: Pipeline + CV + PR-AUC
5. Cierre y entregable

# Lab NumPy — objetivo

**Contexto:** en teoría vimos **generalización**, **fuga de información**, **modelos de referencia** y **métricas** (precisión/recall, exactitud, PR/AP).

**Objetivo del lab:** construir ese workflow **sin librerías "mágicas"**, para entender qué se calcula y cuándo aparece la fuga.

**Al final debes poder:**

- generar un dataset binario sintético $(X, y)$
- hacer un split reproducible train/val/test
- comparar referencia ingenua vs un modelo con **score** + umbral
- medir exactitud, precisión, recall y ver el efecto del umbral
- reproducir una fuga por preprocesamiento (estandarizar "mal" vs "bien")

Archivo: `labs/week01_numpy_baseline.ipynb`

# Lab NumPy — pasos (lo que deben implementar)

1. **Datos (reproducible):** genera $X \in \mathbb{R}^{n \times 2}$, $y \in \{0, 1\}$ (dos nubes / dos gaussianas) y grafica por clase.
2. **Split (sin fuga):** ‘rng.permutation‘ + cortes 60/20/20 en train/val/test (verifica tamaños y proporción de positivos).
3. **Referencia ingenua:** predice la clase mayoritaria (decidida con train) y reporta **exactitud** en val y test.
4. **Score + umbral:** define un score simple $s(x)$ y predice $\hat{y} = \mathbb{1}[s(x) \geq t]$.
5. **Barrido de umbral (clave):** evalúa varios $t$ y arma una tabla en val con: $t$, exactitud, precisión, recall.
6. **Curva PR básica:** con el barrido, grafica puntos (recall, precisión) y elige $t$ según una meta (p. ej. recall mínimo).
7. **Mini-fuga:** estandariza “con todo” vs “solo train” y compara cómo cambian métricas/curva; explica en 2–3 líneas por qué eso es fuga.

**Output esperado:** tabla de umbrales + gráfica PR + comentario corto (elección de $t$ y nota sobre fuga).

# Pausa

Pausa breve

Al volver: **Pipeline + cross-validation + PR-AUC**

# Semana 1 — mapa del temario (progreso)

1. Arranque y framing
2. Teoría: workflow de ML
3. Lab NumPy: baseline + split + métricas
4. Lab sklearn: Pipeline + CV + PR-AUC
5. Cierre y entregable

# Lab sklearn — objetivo

**Objetivo del lab:** implementar un pipeline reproducible en `sklearn` que:

- encapsule el preprocesamiento (evitar fuga),
- evalúe de forma estable (validación cruzada),
- y reporte métricas adecuadas (accuracy y AP / PR-AUC).

**Al final debes poder:**

- construir `Pipeline(StandardScaler, LogisticRegression)`
- evaluar con `cross_val_score` usando:
  - `scoring=.ªccuracy"`
  - `scoring=.ªverage_precision"` (AP / PR-AUC)
- entrenar en train y auditar en test con:
  - `precision_recall_curve`
  - `average_precision_score`

**Regla práctica:** decisiones con validación/CV; test solo auditoría final.
**Archivo:** `labs/week01_sklearn_pipeline_cv.ipynb`

# ¿Qué es cross-validation (CV) y para qué sirve?

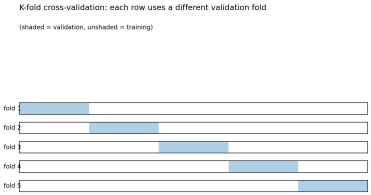
- **Definición:** dividir $\mathcal{D}_{\text{trainval}}$ en $K$ pliegues; repetir $K$ veces: entrenar en $K-1$ y validar en 1; promediar la métrica.
- **Pasos (en práctica):**
  - partir $\mathcal{D}_{\text{trainval}}$ en folds $F_1, \ldots, F_K$
  - para $k = 1, \ldots, K$: `fit` en $\mathcal{D}_{\text{trainval}} \setminus F_k$, evaluar en $F_k$
  - promediar (y reportar variabilidad) de la métrica
  - si hay hiperparámetros $(\lambda, C)$: elegir el que maximiza el promedio en CV
- **Sirve para:** estimar desempeño con menor varianza y **seleccionar** modelo/hiperparámetros (p. ej. $\lambda$, $C$).
- **No reemplaza al test:** el test se guarda para **auditar una sola vez** al final.
- **Cuidado con fuga:** el `Pipeline` asegura que cada pliegue haga `fit` del preprocesamiento solo con su train.

## Regla práctica

Decisiones con CV; test solo auditoría final.

# ¿Por qué cross-validation ayuda? (intuición)

- Con pocos datos, un solo split (train/val) puede dar una métrica **muy variable**.
- K-fold CV promedia el rendimiento sobre $K$ particiones $\Rightarrow$ estimación más **estable**.
- Para selección de modelo/hiperparámetros: elegir el que maximiza la métrica promedio en CV.
- El **test** queda fuera: sirve para auditar el sistema final (sin volver a ajustar).



# Discusión guiada (al final del lab)

## Preguntas

- ¿Cuándo usarías accuracy y cuándo AP/PR-AUC?
- Si hay desbalance, ¿qué te dice AP que no te dice accuracy?
- ¿Qué evita Pipeline que es fácil "romper" si haces preprocessing a mano?

# Semana 1 — mapa del temario (progreso)

1. Arranque y framing
2. Teoría: workflow de ML
3. Lab NumPy: baseline + split + métricas
4. Lab sklearn: Pipeline + CV + PR-AUC
5. Cierre y entregable

# Checklist final (lo que debe quedar bien hoy)

1. Split correcto (sin contaminar test)
2. Baseline reportado (y comparado)
3. Métrica correcta (accuracy + PR/AP)
4. Evitar leakage (Pipeline / fit solo en train)
5. Reproducibilidad (semilla + requirements + README)

# Entregable: repo + notebook plantilla

- Estructura mínima:
  - `slides/week01_intro_workflow.tex`
  - `slides/figs/*.png` (figuras generadas)
  - `labs/week01_numpy_baseline.ipynb`
  - `labs/week01_sklearn_pipeline_cv.ipynb`
  - `README.md` + `requirements.txt` (o `environment.yml`)
- Evidencia mínima:
  - tabla baseline vs logistic (Accuracy + AP)
  - PR curve (imagen) + interpretación (5–6 líneas)

# Cierre: 3 ideas para llevarte

1. **Generalización** es el objetivo; train perfecto no impresiona.
2. **Leakage** es el enemigo silencioso; **Pipeline** es tu escudo.
3. **Baselines + métricas correctas** te mantienen honesto.

# Semana 2 — Probabilidad (modelos univariados) + Bayes

## Bayes, Bernoulli/Binomial, Beta, conjugación y Beta–Binomial (posterior + predictiva)

Curso de Machine Learning — UAQ (Matemáticas Aplicadas)

Libro base: *Probabilistic Machine Learning: An Introduction* (Kevin P. Murphy)

Semana 2

# Lecturas sugeridas (para estos temas)

- **Capítulo 2 — Probability: Univariate Models**
  - **2.3** Bayes' rule
  - **2.4** Bernoulli and binomial distributions
  - **2.7.4** Beta distribution
- **Capítulo 4 — Statistics**
  - **4.6.1** Conjugate priors
  - **4.6.2** The beta-binomial model
  - **4.6.2.4** Posterior
  - **4.6.2.9** Posterior predictive
  - **4.6.2.10** Marginal likelihood

# Temario de la Semana 2 (qué cubrimos hoy)

- **Teoría (PML)**
  - **Teorema de Bayes** y **piezas** (*prior*, *likelihood*, *posterior*, *evidencia*) **(2.3)**
  - **Modelos Bernoulli y Binomial** **(2.4)**
  - **Distribución Beta** (prior sobre probabilidades) **(2.7.4)**
  - **Conjugación** (por qué Beta es conjugada) **(4.6.1)**
  - **Modelo Beta–Binomial**: posterior y posterior predictiva **(4.6.2)**
- **Lab NumPy**
  - **Beta–Bernoulli**: posterior (actualización por conteos)
  - **Posterior predictiva**: 1 ensayo y $M$ ensayos (simulación)
- **Lab sklearn**
  - **Umbral** ($\tau$) y reglas de decisión
  - **Métricas**: Accuracy vs Precision/Recall vs Average Precision
- **Entregable**

# ¿Por qué es relevante el enfoque Bayesiano?

- **Objetivo:** inferir $\theta = \Pr(y = 1)$ (una **probabilidad**) a partir de datos $D$.
- **Incertidumbre explícita:** obtenemos una distribución $p(\theta \mid D)$, no solo un punto.
- **Actualización coherente:** prior + datos $\Rightarrow$ posterior.
- **Predicción robusta:** la **posterior predictiva** integra incertidumbre $\Rightarrow$ menos sobreconfianza con pocos datos.
- **Regularización interpretable:** priors como **pseudoconteos** (suavizado).
- **Comparación de modelos/priors:** evidencia $p(D) =$ **probabilidad marginal de los datos**.

# Enfoque frecuentista (ejemplo: tasa de compra)

- Datos: $n = 20$, $k = 3 \Rightarrow \hat{\theta} = 0{,}15$.
- Estimación: $\hat{\theta} = k/n$ (MLE).
- IC 95 % (aprox. normal):

$$\hat{\theta} \pm 1{,}96\sqrt{\frac{\hat{\theta}(1-\hat{\theta})}{n}} = 0{,}15 \pm 0{,}156 \Rightarrow [-0{,}006,\ 0{,}306]$$

- Intervalo válido en $[0, 1]$: $[0,\ 0{,}306]$.
- Interpretación: 95 % de los ICs (en repeticiones) contienen a $\theta$.

# Enfoque Bayesiano (mismo ejemplo)

- Prior: $\theta \sim \mathrm{Beta}(2, 8)$ (creencia previa $\approx 0{,}20$).
- Posterior: $\theta \mid D \sim \mathrm{Beta}(5, 25)$.
- Media posterior: $\mathbb{E}[\theta \mid D] = \dfrac{5}{30} \approx 0{,}167$.
- Intervalo creíble 95 %: percentiles de $\mathrm{Beta}(5, 25)$.
- Interpretación: $P(\theta \in \mathrm{ICr} \mid D) = 0{,}95$.

# Semana 2 — mapa del temario (progreso)

- **Teoría**
- **Lab NumPy**
- **Lab sklearn**
- **Entregable**

# Motivación: inferir una probabilidad con pocos datos

- Problema base: estimar $\theta = \Pr(y = 1)$ a partir de pocos ensayos Bernoulli.
- Bayes da **distribución** sobre $\theta$, no solo un número.
- Ventaja: incorpora información previa y cuantifica incertidumbre.
- Caso canónico: Bernoulli/Binomial + Beta $\Rightarrow$ actualizaciones cerradas.

# Definiciones (2.3): piezas de Bayes

- **Prior:** $p(\theta)$.
- **Likelihood:** $p(D \mid \theta)$.
- **Posterior:** $p(\theta \mid D) = \dfrac{p(D \mid \theta)\, p(\theta)}{p(D)}$.
- **Evidencia:** $p(D) = \int p(D \mid \theta)\, p(\theta)\, d\theta$ (*marginal likelihood*).

# Derivación 1 (2.4): likelihood Bernoulli/Binomial

- Datos iid $y_i \in \{0, 1\}$, con $k = \sum_{i=1}^{n} y_i$.
- Bernoulli: $p(y_i \mid \theta) = \theta^{y_i}(1-\theta)^{1-y_i}$.
- Likelihood agregada:

$$p(D \mid \theta) = \prod_{i=1}^{n} \theta^{y_i}(1-\theta)^{1-y_i} = \theta^k(1-\theta)^{n-k}.$$

- Binomial: $p(k \mid n, \theta) = \binom{n}{k}\theta^k(1-\theta)^{n-k}$.

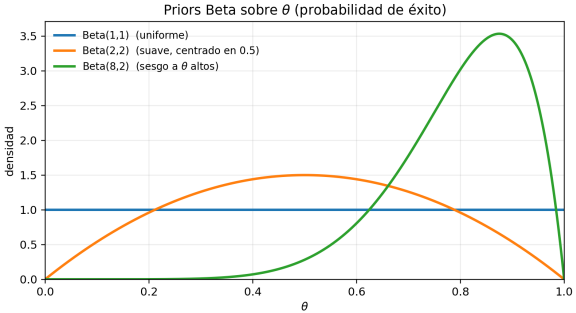
# Distribución Beta (2.7.4): prior sobre probabilidades

- Soporte: $\theta \in (0, 1)$ (ideal para probabilidades).
- Densidad:

$$\theta \sim \mathrm{Beta}(\alpha, \beta), \qquad p(\theta) = \frac{1}{B(\alpha, \beta)} \theta^{\alpha-1} (1-\theta)^{\beta-1}.$$

- Lectura: $\alpha, \beta$ controlan **sesgo** (media) y **fuerza** (concentración).
- Interpretación útil: $\alpha, \beta$ se comportan como **pseudoconteos** de éxitos/fracasos.

# Priors Beta: ¿qué "forma" de creencia estás poniendo?




- $\mathrm{Beta}(1,1)$: uniforme.
- $\mathrm{Beta}(2,2)$: suave, centrada en 0,5.
- $\mathrm{Beta}(8,2)$: empuja hacia $\theta$ altos.
- **Idea:** el prior dice qué valores de $\theta$ son "plausibles" antes de ver datos.

# Derivación 2 (4.6.1): conjugación Beta con Bernoulli/Binomial

- Usamos: $p(\theta \mid D) \propto p(D \mid \theta)\, p(\theta)$.
- Como $p(D \mid \theta) \propto \theta^k (1-\theta)^{n-k}$ y $p(\theta) \propto \theta^{\alpha-1}(1-\theta)^{\beta-1}$, el posterior queda:

$$p(\theta \mid D) \propto \theta^{k+\alpha-1}(1-\theta)^{n-k+\beta-1}.$$

- Conjugación (misma familia):

$$\theta \mid D \sim \mathrm{Beta}(\alpha + k,\ \beta + n - k).$$

- Lectura rápida: $k$ suma a "éxitos" y $n-k$ suma a "fracasos".

# ¿Qué es la **posterior predictiva**?

- **Definición:** distribución de **datos futuros** dado lo observado:

$$p(D_{\text{new}} \mid D) = \int p(D_{\text{new}} \mid \theta)\, p(\theta \mid D)\, d\theta.$$

- **Idea:** en vez de fijar $\theta$ a un punto (plug-in), **promediamos** sobre la incertidumbre en $\theta$.
- En Bernoulli/Binomial:
  - 1 paso: $p(y_{\text{new}} = 1 \mid D) = \mathbb{E}[\theta \mid D]$.
  - $M$ pasos: $S = \sum_{j=1}^{M} y_j \mid D$ tiene una distribución (Beta–Binomial) más "ancha" que plug-in.

# Predictiva *plug-in* vs predictiva Bayesiana (en una línea)

- **Plug-in:** primero estimas $\hat{\theta}$ (p.ej. MLE/MAP/media posterior) y luego predices como si fuera cierto:

$$p_{\text{plug-in}}(D_{\text{new}} \mid D) = p(D_{\text{new}} \mid \hat{\theta}).$$

- **Bayes:** no fijas $\theta$; promedias con el posterior:

$$p(D_{\text{new}} \mid D) = \int p(D_{\text{new}} \mid \theta)\, p(\theta \mid D)\, d\theta.$$

- **Mensaje:** plug-in suele ser más "optimista"; Bayes tiende a ser menos sobreconfiante con pocos datos.

# Modelo Beta–Binomial (4.6.2): posterior y predictiva

- Predictiva 1 paso:

$$p(y_{\text{nuevo}} = 1 \mid D) = \mathbb{E}[\theta \mid D] = \frac{\alpha + k}{\alpha + \beta + n}.$$

- Predictiva de conteos ($m$ futuros):

$$p(k_{\text{new}} \mid m, D) = \binom{m}{k_{\text{new}}} \frac{B(\alpha + k + k_{\text{new}}, \beta + n - k + m - k_{\text{new}})}{B(\alpha + k, \beta + n - k)}.$$

# Ejemplo numérico mínimo

- Prior uniforme: $\theta \sim \mathrm{Beta}(1, 1)$.
- Datos: $n = 4$ lanzamientos, $k = 3$ éxitos.
- Posterior: $\theta \mid D \sim \mathrm{Beta}(4, 2)$.
- Predictiva: $p(y_{\text{nuevo}} = 1 \mid D) = \dfrac{4}{6} = \dfrac{2}{3}$.

# Derivación (4.6.2.10): evidencia Beta–Binomial $p(k \mid n, \alpha, \beta)$

- Definición (marginalizar $\theta$):

$$p(k \mid n, \alpha, \beta) = \int_0^1 p(k \mid n, \theta)\, p(\theta \mid \alpha, \beta)\, d\theta.$$

- Sustituimos Binomial y Beta:

$$p(k \mid n, \theta) = \binom{n}{k} \theta^k (1-\theta)^{n-k}, \quad p(\theta) = \frac{1}{B(\alpha, \beta)} \theta^{\alpha-1} (1-\theta)^{\beta-1}.$$

- Agrupando potencias:

$$p(k \mid n, \alpha, \beta) = \binom{n}{k} \frac{1}{B(\alpha, \beta)} \int_0^1 \theta^{\alpha+k-1} (1-\theta)^{\beta+n-k-1}\, d\theta.$$

- Reconocemos la integral como $B(\alpha + k, \beta + n - k) \Rightarrow$

$$p(k \mid n, \alpha, \beta) = \binom{n}{k} \frac{B(\alpha + k, \beta + n - k)}{B(\alpha, \beta)}.$$

# Evidencia $p(D)$: intuición y comparación de modelos

- **Qué es:** $p(D)$ es la **probabilidad marginal** del dataset (también *marginal likelihood*).
- **Rol en Bayes:** es el **normalizador** (no depende de $\theta$).
- **Intuición:** promedia el likelihood sobre $\theta$ "plausibles" según el prior.
- **Comparación:** para dos modelos/priors $M_1, M_2$,

$$\text{Bayes factor} = \frac{p(D \mid M_1)}{p(D \mid M_2)}.$$

- **Lectura:** favorece modelos que explican bien los datos **sin** "apostar" probabilidad en demasiados $\theta$.

# Evidencia: misma data, distinto prior $\Rightarrow$ distinto $p(D)$

- Datos (Binomial): $n = 10$, $k = 7$ éxitos.
- Con prior $\theta \sim \mathrm{Beta}(\alpha, \beta)$:

$$p(k \mid n, \alpha, \beta) = \binom{n}{k} \frac{B(\alpha + k, \beta + n - k)}{B(\alpha, \beta)}.$$

- Evidencias (aprox.):
  - $\mathrm{Beta}(1, 1)$: $p(k \mid n) \approx 0{,}0909$
  - $\mathrm{Beta}(2, 2)$: $p(k \mid n) \approx 0{,}1119$
  - $\mathrm{Beta}(8, 2)$: $p(k \mid n) \approx 0{,}1486$
- **Interpretación (en simple):** estos números dicen qué tan "esperable" era ver $7/10$ *antes* de ver los datos, según cada prior.
- **Lo que importa: comparaciones.** Por ejemplo:

$$\frac{p(D \mid \mathrm{Beta}(8, 2))}{p(D \mid \mathrm{Beta}(1, 1))} \approx 1{,}63, \qquad \frac{p(D \mid \mathrm{Beta}(2, 2))}{p(D \mid \mathrm{Beta}(1, 1))} \approx 1{,}23.$$

# Resumen y takeaways

- Bayes (2.3) combina prior y likelihood para obtener $p(\theta \mid D)$.
- Bernoulli/Binomial (2.4) $\Rightarrow$ likelihood $\theta^k(1-\theta)^{n-k}$.
- Beta (2.7.4) $\Rightarrow$ prior natural sobre probabilidades.
- Conjugación (4.6.1): $\mathrm{Beta}(\alpha, \beta) \to \mathrm{Beta}(\alpha + k, \beta + n - k)$.
- Predictiva (4.6.2): promedia sobre $\theta \Rightarrow$ menos sobreconfianza.
- Evidencia (4.6.2.10): $p(D)$ sirve para **comparar** priors/modelos (Bayes factor).

# Semana 2 — mapa del temario (progreso)

- **Teoría**
- **Lab NumPy**
- **Lab sklearn**
- **Entregable**

# Lab NumPy — objetivos (qué debes lograr)

- **Inferencia:** a partir de $y \in \{0,1\}^n$, calcular $N_1, N_0$ y actualizar $(\alpha, \beta) \to (\alpha', \beta')$.
- **Predicción:** calcular $p(y_{\text{new}} = 1 \mid D)$ y aproximar $p(S \mid D)$ para $M$ futuros.
- **Comparación:** contrastar predictiva *plug-in* vs *Bayesiana* y explicar el efecto del prior (sesgo y fuerza) en 2–3 frases.

# Lab NumPy — definiciones mínimas (puente con Teoría)

- **Parámetro:** $\theta = \Pr(y = 1)$.
- **Datos:** $D = \{y_i\}_{i=1}^{n}$, con conteos $N_1 = \sum_i y_i$, $N_0 = n - N_1$.
- **Prior:** $\theta \sim \mathrm{Beta}(\alpha, \beta)$.
- **Posterior (conjugación):** $\theta \mid D \sim \mathrm{Beta}(\alpha', \beta')$, donde $\alpha' = \alpha + N_1$, $\beta' = \beta + N_0$.
- **Posterior predictiva:** $p(D_{\text{new}} \mid D) = \int p(D_{\text{new}} \mid \theta) p(\theta \mid D) d\theta$.
- **Plug-in vs Bayes:** plug-in fija $\theta = \hat{\theta}$; Bayes integra sobre $\theta$.

# Lab NumPy — Pregunta 1: ¿Qué datos tengo y qué resumen necesito?

```
ENTRADA: y[1..n]  (valores 0/1)

N1 <- suma(y)
N0 <- n - N1

SALIDA: N1, N0
CHEQUEO: N1 + N0 = n
```

- Definición: $N_1$ = número de éxitos, $N_0$ = número de fracasos.

# Lab NumPy — Pregunta 2: ¿Qué creía antes de ver datos? (prior)

```
ELIGE el prior (alpha > 0, beta > 0)

EJEMPLOS:
  alpha = 1, beta = 1   (uniforme)
  alpha = 2, beta = 2   (suave, centrado en 0.5)
  alpha = 8, beta = 2   (sesgo hacia theta altos)
```

- Tip: $\alpha + \beta$ controla la "fuerza" del prior.
- Pitfall: prior fuerte sin justificar.

# Lab NumPy — Pregunta 3: ¿Qué creo después de ver datos? (posterior)

```
ACTUALIZA (conjugacion Beta--Bernoulli):
alpha_post <- alpha + N1
beta_post  <- beta  + N0

SALIDA: alpha_post, beta_post
CHEQUEO: alpha_post > 0 y beta_post > 0
```

- Definición: $\alpha' = \alpha + N_1$, $\beta' = \beta + N_0$.

# Lab NumPy — Pregunta 4: ¿Qué predigo del siguiente ensayo? (predictiva 1 paso)

```
PREDICTIVA POSTERIOR (1 ensayo):
p1_bayes <- alpha_post / (alpha_post + beta_post)
p0_bayes <- 1 - p1_bayes

SALIDA: p(y_nuevo = 1 | D) = p1_bayes
```

- Definición: $p(y_{\text{new}} = 1 \mid D) = \mathbb{E}[\theta \mid D] = \dfrac{\alpha'}{\alpha' + \beta'}$.

# Lab NumPy — Pregunta 5: ¿Qué predigo de $M$ ensayos futuros? (predictiva de conteos)

```
FIJA M (ensayos futuros) y R (numero de simulaciones)

PARA r = 1..R:
  muestrea theta_r ~ Beta(alpha_post, beta_post)
  muestrea S_r ~ Binomial(M, theta_r)

ESTIMA p(S | D) con {S_r} (histograma / frecuencias)
CHEQUEO: suma_s p_hat(S=s | D) ~= 1
```

- Definición: $S = \sum_{j=1}^{M} y_j =$ número de éxitos futuros en $M$ ensayos.
- Lógica Bayesiana: muestrear $\theta \sim \mathrm{Beta}(\alpha', \beta')$ y luego $S \sim \mathrm{Binomial}(M, \theta)$.

# Lab NumPy — Pregunta 6: Predictiva plug-in vs Bayesiana

**Objetivo:** Comparar dos formas de predecir $S$ = número de éxitos en $M$ ensayos futuros.

| | **S_bayes** | **S_plugin** |
|---|---|---|
| Qué es | Muestras de $p(S \mid D)$ | Muestras de $\mathrm{Bin}(M, \hat{\theta})$ |
| Cómo se obtiene | $\theta \sim \mathrm{Beta}(\alpha', \beta')$, luego $S \sim \mathrm{Bin}(M, \theta)$ | $S \sim \mathrm{Bin}(M, \theta_{\mathrm{MAP}})$ con $\theta$ fijo |
| Incertidumbre sobre $\theta$ | Sí (integra sobre la posterior) | No (usa un solo $\hat{\theta}$) |

**Pasos:**

1. **Estimador puntual:** $\theta_{\mathrm{MAP}}$. Si $\alpha' > 1$ y $\beta' > 1$: $\dfrac{\alpha' - 1}{\alpha' + \beta' - 2}$; si no: $\dfrac{\alpha'}{\alpha' + \beta'}$.
2. **S_plugin:** para $r = 1, \ldots, R$, muestrea $S_r \sim \mathrm{Binomial}(M, \theta_{\mathrm{MAP}})$.
3. **S_bayes:** para $r = 1, \ldots, R$, muestrea $\theta_r \sim \mathrm{Beta}(\alpha', \beta')$, luego $S_r \sim \mathrm{Binomial}(M, \theta_r)$.
4. **Comparar:** media, varianza y una probabilidad de cola (ej. $P(S \leq k)$ o $P(S \geq k)$) de S_plugin vs S_bayes. Opcional: histogramas superpuestos.

**Interpretación:** S_bayes suele tener más varianza y colas más pesadas; el plug-in suele subestimar la

# Lab NumPy — entregables (checklist)

Tu notebook debe incluir (evaluable):

- **Resultados:** $(\alpha, \beta)$, $(\alpha', \beta')$, $N_1, N_0$, y $p(y_{\text{new}} = 1 \mid D)$.
- **Predictiva:** aproximación de $p(S \mid D)$ para algún $M$ (elige $M$ y $R$ y repórtalos).
- **Comparación:** Bayes vs plug-in (una figura o, mínimo, media/varianza y una probabilidad de cola).
- **Discusión (2–3 frases):** qué cambia al variar $\alpha + \beta$ (fuerza) y la media del prior (sesgo).
- **Reproducible:** semilla fija y celdas sin salidas gigantes.

# Semana 2 — mapa del temario (progreso)

- **Teoría**
- **Lab NumPy**
- **Lab sklearn**
- **Entregable**

# Lab sklearn — objetivo

En un problema binario:

- entrenar un modelo que produzca $\hat{p}$
- barrer umbrales $\tau$ y ver trade-offs
- comparar accuracy vs precision/recall vs average precision

# Lab sklearn — Paso 1: modelo con probabilidades

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# X, y: tus datos (numpy arrays)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0)

clf = LogisticRegression(max_iter=1000)
clf.fit(Xtr, ytr)

p_hat = clf.predict_proba(Xte)[:, 1]  # scores en [0,1]
```

# Lab sklearn — Paso 2: umbral y conteos TP/FP/TN/FN

```
def yhat_from_tau(p, tau):
    return (p >= tau).astype(int)

tau = 0.5
yhat = yhat_from_tau(p_hat, tau)

TP = int(((yhat==1) & (yte==1)).sum())
FP = int(((yhat==1) & (yte==0)).sum())
TN = int(((yhat==0) & (yte==0)).sum())
FN = int(((yhat==0) & (yte==1)).sum())
print(TP, FP, TN, FN)
```

# Lab sklearn — Paso 3: métricas por umbral

```
from sklearn.metrics import accuracy_score, precision_score, recall_score

taus = np.linspace(0.0, 1.0, 101)
rows = []
for tau in taus:
    yhat = (p_hat >= tau).astype(int)
    acc = accuracy_score(yte, yhat)
    prec = precision_score(yte, yhat, zero_division=0)
    rec  = recall_score(yte, yhat, zero_division=0)
    rows.append([tau, acc, prec, rec])

rows = np.array(rows)
best_tau_acc = rows[rows[:,1].argmax(), 0]
print("best tau (accuracy):", best_tau_acc)
```

- Pitfall: escoger $\tau$ usando el test (se elige con validación).

# Lab sklearn — Paso 4: Average Precision (AP) y curva PR

```
from sklearn.metrics import average_precision_score, precision_recall_curve

ap = average_precision_score(yte, p_hat)
prec, rec, thr = precision_recall_curve(yte, p_hat)

print("Average Precision =", ap)
# (Opcional) graficar: rec (x) vs prec (y)
```

# Lab sklearn — preguntas guía

- ¿Qué pasa con precision y recall cuando subes $\tau$?
- ¿Por qué accuracy puede ser engañosa con desbalance?
- ¿Qué te dice AP que no te dice un umbral fijo?

# Semana 2 — mapa del temario (progreso)

- **Teoría**
- **Lab NumPy**
- **Lab sklearn**
- **Entregable**

# Entregable (Semana 2)

Subir al repo:

- **Notebook 1 (NumPy):** Beta–Bernoulli posterior + posterior predictiva (1 y $M$).
- **Notebook 2 (sklearn):** umbrales + tabla de métricas + AP (y opcional curva PR).
- **README corto:** cómo correr, qué aprendiste (5–8 líneas) y una figura/tabla destacada.

# Checklist de entrega (formato)

- Carpeta: `week02/`
- Nombres: `week02_beta_bernoulli.ipynb`, `week02_thresholding_metrics.ipynb`
- Celdas limpias (sin outputs gigantes) y semilla fija donde aplique.
- Commits pequeños con mensajes claros.
- Reproducible: `requirements.txt` o `environment.yml` (si ya lo estamos usando).

# Cierre

- Bayes = prior $\times$ likelihood $\rightarrow$ posterior (actualización).
- Conjugación = actualizaciones simples (conteos $\rightarrow$ parámetros).
- Predicción Bayesiana integra incertidumbre $\Rightarrow$ menos sobreconfianza.
- En clasificación: el score no es la decisión; la decisión depende del umbral y del objetivo.

# Semana 3 — Probabilidad (modelos multivariados)

De describir dependencia a inferir faltantes: $\Sigma/R \to$ MVN $\to$ Mahalanobis $\to$ condicionamiento $\to$ imputación (Iris) $\to$ GMM

Curso de Machine Learning — UAQ (Matemáticas Aplicadas)

Libro base: *Probabilistic Machine Learning: An Introduction* (Kevin P. Murphy)

Semana 3

# Lecturas sugeridas (para estos temas)

- **Capítulo 3 — Probability: Multivariate Models**
  - **3.1.1** Covarianza
  - **3.1.2** Correlación
  - **3.1.3** No correlación $\not\Rightarrow$ independencia
  - **3.1.4** Correlación $\not\Rightarrow$ causalidad
  - **3.1.5** Paradoja de Simpson
  - **3.2.1** Definición de la MVN
  - **3.2.2** Distancia de Mahalanobis
  - **3.2.3** Marginales y condicionales de una MVN *
  - **3.2.4** Ejemplo: condicionamiento en una Gaussiana 2D
  - **3.2.5** Ejemplo: imputación de valores faltantes *
  - **3.3.1** Regla de Bayes para gaussianas
  - **3.5.1** Modelos de mezcla gaussianos (GMM)

# Temario de la Semana 3 (qué cubrimos hoy)

- **Teoría (PML)**
  - **1) Describir dependencia:** $\hat{\Sigma}$ y $\hat{R}$ **(3.1.1–3.1.2)**
  - **2) Interpretar con cuidado:** $\rho = 0 \not\Rightarrow$ independencia; correlación $\not\Rightarrow$ causalidad; Simpson **(3.1.3–3.1.5)**
  - **3) Modelar:** MVN $\mathcal{N}(\mu, \Sigma)$ como baseline **(3.2.1)**
  - **4) Geometría/probabilidad:** Mahalanobis (qué tan "raro" es un punto) **(3.2.2)**
  - **5) Inferir:** marginales/condicionales y condicionamiento por bloques **(3.2.3)**
  - **6) Aplicación:** imputación (Iris) como $\mathbb{E}[X_m \mid X_o]$ **(3.2.5)**
  - **7) Extensión:** GMM para multimodalidad **(3.5.1)**
- **Lab NumPy**
  - **Condicionamiento en una MVN** (bloques + fórmulas)
  - **Imputación:** $\mathbb{E}[x_{\text{miss}} \mid x_{\text{obs}}]$
  - **Evaluación:** MSE en valores ocultados
- **Lab sklearn**
  - **GaussianMixture:** ajustar un GMM en 2D
  - **Leer parámetros:** weights, means, covariances
  - **Visualización:** puntos + elipses por componente
- **Entregable**

# Hilo conductor de la semana (Iris)

- Vamos a usar **Iris** como ejemplo mental: 4 variables (sépalo/pétalo).
- **Pregunta 1 (describir):** ¿qué variables se mueven juntas? $\Rightarrow \hat{\Sigma}$, $\hat{R}$.
- **Pregunta 2 (modelar):** si aproximamos con una MVN, ¿cómo se ve la nube? $\Rightarrow$ geometría de $\Sigma$.
- **Pregunta 3 (medir rareza):** ¿qué puntos son atípicos bajo ese modelo? $\Rightarrow$ Mahalanobis.
- **Pregunta 4 (inferir faltantes):** si falta una variable, ¿qué valor es más plausible? $\Rightarrow$ condicionamiento $\rightarrow$ imputación.
- **Pregunta 5 (ir más allá):** si hay varias "nubes", ¿cómo lo modelamos? $\Rightarrow$ GMM.

# Semana 3 — mapa del temario (progreso)

- **Teoría**
- **Lab NumPy**
- **Lab sklearn**
- **Entregable**

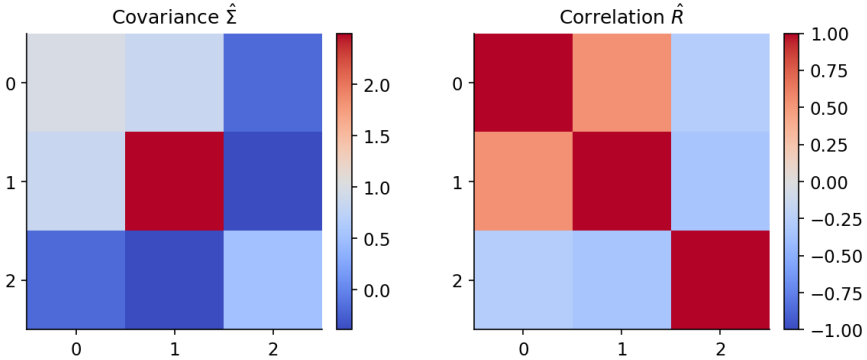
# Punto de partida: vector aleatorio, media y covarianza

- En datos reales (p. ej. Iris), cada fila es un vector $x \in \mathbb{R}^D$ (aquí $D = 4$).
- Modelamos $X$ como vector aleatorio: $X \in \mathbb{R}^D$.
- **Media**: $\mu = \mathbb{E}[X]$ (valor "típico" por variable).
- **Covarianza**: $\Sigma = \mathbb{E}[(X - \mu)(X - \mu)^\top]$ (escala y dependencia lineal).
- **Entrada**: $\Sigma_{ij} = \mathrm{Cov}(X_i, X_j)$ indica si dos variables tienden a subir/bajar juntas.

# Construcción en práctica: $\hat{\Sigma}$ y $\hat{R}$ (pasos)

1. **Paso 1 — datos:** Tenemos $n$ observaciones $x_1, \ldots, x_n$ (filas). Puede ser Iris (4 variables) o un ejemplo 3D.
2. **Paso 2 — covarianza muestral:** Estimamos $\hat{\Sigma}_{ij}$ como el promedio centrado de productos de desviaciones:
$$\hat{\Sigma}_{ij} = \frac{1}{n-1} \sum_{k=1}^{n} (x_{ki} - \bar{x}_i)(x_{kj} - \bar{x}_j), \quad \bar{x}_i = \frac{1}{n} \sum_{k=1}^{n} x_{ki}.$$
Diagonal = varianza muestral; fuera de diagonal = covarianza (nota: *unidades mezcladas*).
3. **Paso 3 — correlación:** Estandarizamos a escala $[-1, 1]$:
$$\hat{R}_{ij} = \frac{\hat{\Sigma}_{ij}}{\sqrt{\hat{\Sigma}_{ii}\hat{\Sigma}_{jj}}}.$$
4. **Lectura rápida:** usa $\hat{R}$ para comparar dependencias; usa $\hat{\Sigma}$ para cálculos (geometría y condicionamiento).

# $\hat{\Sigma}$ vs $\hat{R}$: mismo dato, dos "lentes"

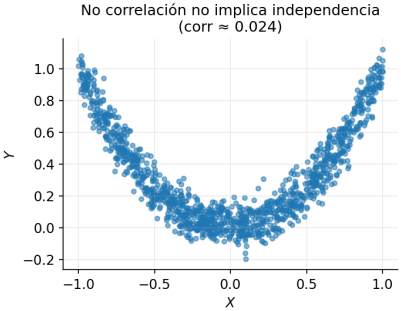



**Idea:** la izquierda conserva escala (varianzas grandes dominan); la derecha estandariza y hace comparables las relaciones lineales.

Izquierda: $\hat{\Sigma}$. Derecha: $\hat{R}$.

# Alerta 1: $\rho = 0$ no implica independencia

- **Mensaje**: correlación mide *dependencia lineal*. Puede haber relación no lineal con $\rho \approx 0$.
- **Hecho**: $\mathrm{Cov}(X, Y) = 0 \not\Rightarrow X \perp Y$ (salvo casos especiales como gaussianas).
- **Traducción práctica**: "no hay correlación" $\neq$ "no hay relación".



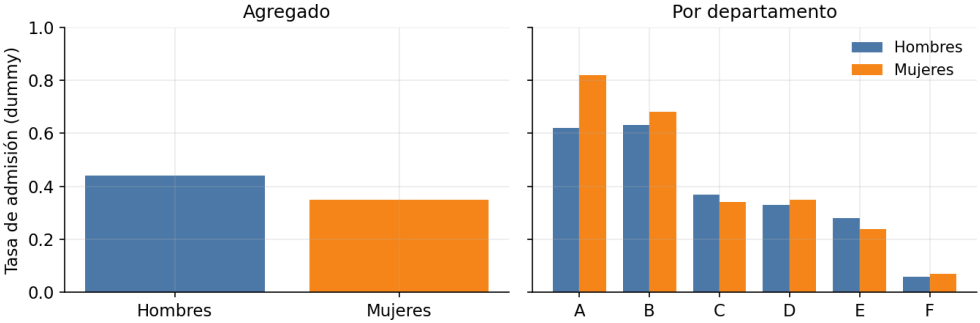
# Alerta 2: correlación no implica causalidad

- Una correlación puede venir de **confusores**, **sesgo de selección** o **causalidad invertida**.
- **Pregunta guía**: ¿qué variable podría estar afectando a ambas? ¿qué estoy dejando fuera?

# Paradoja de Simpson: cuando el agregado engaña

- **Definición**: una tendencia en datos **agregados** cambia (o se invierte) al **estratificar** por una tercera variable.
- **Ejemplo (Berkeley, 1973)**: al controlar por departamento, la interpretación cambia.
  - **Agregado**: parece haber sesgo.
  - **Estratificado**: aparece el rol del confusor (departamento).

# Paradoja de Simpson: visualización y “checklist mental”



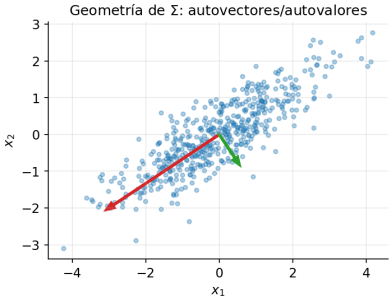

- **Checklist**: antes de concluir, pregunta “¿qué variable podría estar mezclando grupos?”.
- Ejemplos de estratos: departamento, cohorte, nivel educativo, hospital/escuela, etc.

# MVN: un baseline probabilístico para vectores

- Idea: aproximar una nube de datos con una sola *elipse* (2D) / *elipsoide* (D).
- Notación: $X \sim \mathcal{N}(\mu, \Sigma)$.
- $\mu$: centro; $\Sigma$: escala + orientación (forma).
- Esto nos da dos cosas: **geometría** (elipses) y **cálculo cerrado** (condicionales).

# Geometría de $\Sigma$: cómo "lee" la MVN la nube de datos

- En una MVN, las curvas de igual densidad son *elipses*/*elipsoides*.
- **Autovectores**: direcciones principales (ejes) de la nube.
- **Autovalores**: dispersión en cada dirección (eje $\propto \sqrt{\lambda}$).
- **Lectura práctica**: $\Sigma$ codifica *escala* y *correlación* en una sola matriz.

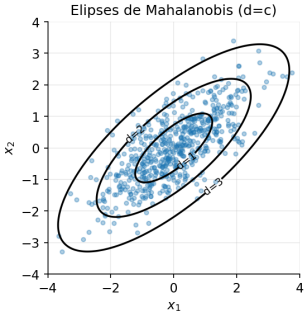


# Distancia de Mahalanobis: qué es y para qué sirve

- **Definición:**

$$d_M(x,\mu) = \sqrt{(x-\mu)^\top \Sigma^{-1}(x-\mu)}.$$

- **Intuición:** mide "lejanía" en unidades que respetan *escala* y *correlación*.
- **Sirve para:**
  - detectar *atípicos* bajo una MVN (outliers),
  - entender el *log-likelihood* gaussiano (el término cuadrático),
  - conectar con GMM: cada componente usa su $\Sigma_k$ para medir cercanía.
- **Puente:** la geometría que define $d_M$ es la misma que usamos al **condicionar** (siguiente bloque).



# De “medir rareza” a “inferir faltantes”

- Con $\mu$ y $\Sigma$ ya podemos:
  - evaluar qué tan plausible es un punto (vía $d_M$),
  - y también **inferir** partes faltantes de un vector.
- La herramienta clave es que en la MVN, **marginales** y **condicionales** siguen siendo gaussianas.

# Marginales y condicionales en una MVN (la pieza que habilita todo)

- **Marginal**: si te quedas con algunas coordenadas, sigues en el mundo gaussiano.
- **Condicional**: si conoces parte del vector, la distribución del resto también es gaussiana.
- **Consecuencia**: podemos calcular $\mathbb{E}[X_{\text{faltante}} \mid X_{\text{observado}}]$ en forma cerrada $\Rightarrow$ imputación.

# Condicionamiento por bloques: la receta (MVN)

- **Notación**: $a$ = variables *observadas*; $b$ = variables *faltantes* (o las que queremos predecir).
- Partición:

$$x = \begin{pmatrix} x_a \\ x_b \end{pmatrix}, \quad \mu = \begin{pmatrix} \mu_a \\ \mu_b \end{pmatrix}, \quad \Sigma = \begin{pmatrix} \Sigma_{aa} & \Sigma_{ab} \\ \Sigma_{ba} & \Sigma_{bb} \end{pmatrix}.$$

- Resultado: $X_b \mid X_a = a \sim \mathcal{N}(\mu_{b|a}, \Sigma_{b|a})$.
- Fórmulas:

$$\mu_{b|a} = \mu_b + \Sigma_{ba}\Sigma_{aa}^{-1}(a - \mu_a), \qquad \Sigma_{b|a} = \Sigma_{bb} - \Sigma_{ba}\Sigma_{aa}^{-1}\Sigma_{ab}.$$

- Nota: $\Sigma_{b|a}$ no depende de $a$. Para imputación "simple", usamos la media $\mu_{b|a}$.

# Condicionamiento por bloques: ejemplo numérico (D=5)

**Ejemplo:** observamos 3 variables ($a$) y faltan 2 ($b$). Así "se ven" $\mu$ y $\Sigma$.

- Vector de medias: $\mu = \begin{pmatrix} 2{,}1 \\ 1{,}8 \\ 3{,}0 \\ 4{,}2 \\ 5{,}5 \end{pmatrix}$ (shape $(5,)$).
- Matriz de covarianza $\Sigma$ (shape $(5 \times 5)$):

$$\Sigma = \begin{pmatrix} 1{,}2 & 0{,}5 & 0{,}3 & 0{,}4 & 0{,}2 \\ 0{,}5 & 0{,}9 & 0{,}2 & 0{,}3 & 0{,}1 \\ 0{,}3 & 0{,}2 & 1{,}5 & 0{,}6 & 0{,}4 \\ 0{,}4 & 0{,}3 & 0{,}6 & 2{,}0 & 0{,}8 \\ 0{,}2 & 0{,}1 & 0{,}4 & 0{,}8 & 1{,}8 \end{pmatrix}$$

- Partición: $a =$ índices $[0, 1, 2]$ (observadas), $b =$ índices $[3, 4]$ (faltantes).

# Condicionamiento por bloques: ejemplo numérico (D=5) — subbloques

- **Partir** $\mu$ en observado/faltante:

$$\mu_a = \begin{pmatrix} 2{,}1 \\ 1{,}8 \\ 3{,}0 \end{pmatrix} \quad \text{(shape } (3,)\text{)}, \qquad \mu_b = \begin{pmatrix} 4{,}2 \\ 5{,}5 \end{pmatrix} \quad \text{(shape } (2,)\text{)}.$$

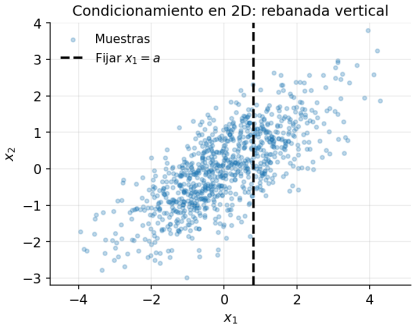
- **Partir** $\Sigma$ en 4 bloques (observado-observado, observado-faltante, etc.):

$$\Sigma_{aa} = \begin{pmatrix} 1{,}2 & 0{,}5 & 0{,}3 \\ 0{,}5 & 0{,}9 & 0{,}2 \\ 0{,}3 & 0{,}2 & 1{,}5 \end{pmatrix} \quad \text{(shape } (3 \times 3)\text{)},$$

$$\Sigma_{ab} = \begin{pmatrix} 0{,}4 & 0{,}2 \\ 0{,}3 & 0{,}1 \\ 0{,}6 & 0{,}4 \end{pmatrix} \quad \text{(shape } (3 \times 2)\text{)}, \qquad \Sigma_{ba} = \Sigma_{ab}^{\top}, \quad \Sigma_{bb} = \begin{pmatrix} 2{,}0 & 0{,}8 \\ 0{,}8 & 1{,}8 \end{pmatrix} \quad \text{(shape } (2 \times 2)\text{)}.$$

- **Lectura**: $\Sigma_{aa}$ es lo que invertimos; $\Sigma_{ba}$ "transporta" información de observado $\rightarrow$ faltante.
- Para imputar: $\mu_{b|a} = \mu_b + \Sigma_{ba}\Sigma_{aa}^{-1}(x_a - \mu_a)$ (shape $(2,)$).

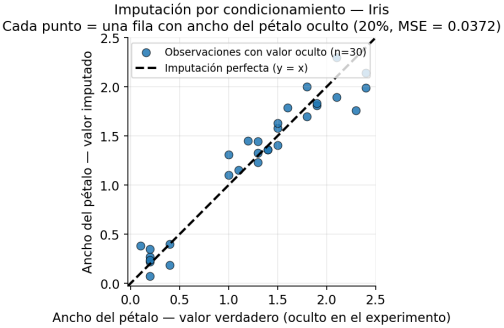
# Condicionamiento en 2D: intuición visual (rebanar la elipse)




- En 2D: observado $a = x_1$, faltante $b = x_2$.
- Fijar $x_1 = a$ "rebana" la nube: queda una normal 1D para $x_2 \mid x_1 = a$.

# Imputación (Iris) = condicionamiento + evaluación honesta

- **Idea**: tratar "faltante" como $X_m$ y lo observado como $X_o$.
- **Regla**: imputación determinista $x_m \leftarrow \mathbb{E}[X_m \mid X_o = x_o]$ (media condicional).
- **Experimento**: ocultamos ancho del pétalo en 20 % de filas; estimamos $(\mu, \Sigma)$ con las filas observadas; imputamos y comparamos.
- **Gráfica**: cada punto azul = una celda ocultada (verdadero vs imputado). Diagonal $y = x$ = ideal.
- **Métrica**: MSE **solo** donde ocultaste; compara contra baseline (media marginal por columna).



# ¿Por qué no basta una sola MVN? (puente a mixtures)

- Una MVN = una sola "nube elíptica".
- En Iris, distintas especies sugieren **más de una nube** (multimodalidad).
- Solución: **mezcla** = suma ponderada de varias MVNs.

# GMM: varias elipses para explicar una nube compleja

- Modelo:

$$p(x) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k), \qquad \sum_k \pi_k = 1,\ \pi_k \geq 0.$$

- En 2D, cada componente induce una elipse; el modelo suma "nubes".
- Interpretación: $\pi_k$ (peso), $\mu_k$ (centro), $\Sigma_k$ (forma).
- Conexión: "cercanía" a cada componente se relaciona con Mahalanobis y su log-likelihood.

# Cierre: qué te llevas de Week03

- $\hat{\Sigma}$ conserva escala; $\hat{R}$ estandariza para comparar dependencias.
- Interpretación cuidadosa: $\rho = 0 \not\Rightarrow$ independencia; correlación $\not\Rightarrow$ causalidad; cuidado con Simpson.
- MVN + $\Sigma$ da geometría; Mahalanobis mide "rareza" bajo esa geometría.
- Condicionamiento MVN es cerrado $\Rightarrow$ imputación como $\mathbb{E}[X_m \mid X_o]$.
- Si hay varias nubes, GMM (mezcla de elipses) es una extensión natural.

# Semana 3 — mapa del temario (progreso)

- **Teoría**
- **Lab NumPy**
- **Lab sklearn**
- **Entregable**

# Lab NumPy — objetivo general

- Implementar la **media condicional** de una MVN por bloques (la receta de Teoría).
- Aplicarla a **imputación** (p. ej. Iris: ocultar una columna y recuperarla).
- Evaluar sin engañarse: MSE **solo** en valores ocultados + baseline.
- Reproducibilidad: semilla fija, reporta $N$ y % faltantes.

# Lab NumPy — notación y shapes (traducción Teoría → código)

- `mu (D,)`: vector de medias; `mu[i]` = media de la variable $i$.
- `Sigma (D,D)`: covarianza; `Sigma[i,j]` = $\mathrm{cov}(X_i, X_j)$.
- `idx_a / idx_b`: índices de observado/faltante (en Iris, $D = 4$ y puedes ocultar "petal width").
- `x_a`: valores observados de una fila; es el "$a$" numérico de esa observación.
- Fórmula por fila (media condicional): $\mu_{b|a} = \mu_b + \Sigma_{ba}\Sigma_{aa}^{-1}(x_a - \mu_a)$.

# Lab NumPy — Pseudocódigo 1: partir en bloques

```
# INPUT:
#   mu:    shape (D,)    # vector de medias
#   Sigma: shape (D,D)   # matriz de covarianza
#   idx_a: índices observados
#   idx_b: índices faltantes

mu_a = mu[idx_a]                          # (Da,)
mu_b = mu[idx_b]                          # (Db,)
S_aa = Sigma[np.ix_(idx_a, idx_a)]        # (Da,Da)
S_ab = Sigma[np.ix_(idx_a, idx_b)]        # (Da,Db)
S_ba = Sigma[np.ix_(idx_b, idx_a)]        # (Db,Da)
S_bb = Sigma[np.ix_(idx_b, idx_b)]        # (Db,Db)

# CHEQUEO: S_aa debe ser invertible (si no, regulariza: S_aa += eps*I)
```

# Lab NumPy — Pseudocódigo 2: media condicional → imputación

```
# Precompute una vez (fuera del loop por filas):
S_aa_inv = inv(S_aa)   # costoso -> hazlo una sola vez

# Para cada fila i con faltantes en idx_b:
x_a = X[i, idx_a]      # valores observados de la fila (Da,)

# Fórmula MVN:
# mu_{b|a} = mu_b + S_ba @ inv(S_aa) @ (x_a - mu_a)
mu_b_given_a = mu_b + S_ba @ S_aa_inv @ (x_a - mu_a)  # (Db,)

# Imputación determinista (media condicional):
X_imputed[i, idx_b] = mu_b_given_a

# Opcional (imputación estocástica):
# S_{b|a} = S_bb - S_ba @ inv(S_aa) @ S_ab
```

# Lab NumPy — Pseudocódigo 3: evaluación correcta (MSE)

```
# 1) Guarda el dato completo
X_true = X.copy()                    # (N,D)

# 2) Genera máscara de celdas a ocultar (True = oculto)
mask = make_mask(...)                # (N,D) boolean

# 3) Oculta y luego imputa
X_missing = X_true.copy()
X_missing[mask] = NaN
X_imputed = impute(X_missing)        # tu función del pseudocódigo 2

# 4) Evalúa solo donde ocultaste (para no "hacer trampa")
mse = mean( (X_true[mask] - X_imputed[mask])**2 )

# Baseline: imputar por media marginal (media de columna)
```

# Semana 3 — mapa del temario (progreso)

- **Teoría**
- **Lab NumPy**
- **Lab sklearn**
- **Entregable**

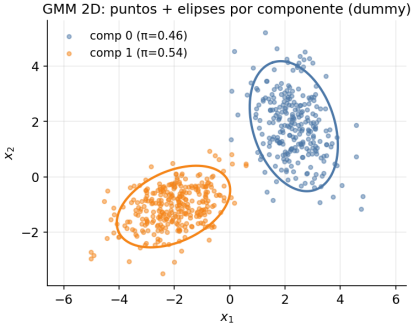
# Lab sklearn — objetivo general

- Ajustar un **GMM** en 2D (elige $K$, fija semilla).
- Leer parámetros: pesos, medias y covarianzas $\Rightarrow$ "varias elipses".
- Visualizar: puntos + elipses por componente.
- Interpretar: 2–3 frases (¿qué parte de la nube captura cada componente?).

# GMM: cómo interpretar $\pi_k, \mu_k, \Sigma_k$

- $\pi_k$: proporción de datos que "explica" el componente $k$.
- $\mu_k$: centro del componente.
- $\Sigma_k$: forma/orientación (elipse) del componente.
- **Pregunta guía:** ¿el componente captura estructura real o está "partiendo" artificialmente una sola nube?

# GMM en 2D: visualización (puntos + elipses)




- Elipses = covarianza por componente; colores = componente más probable.
- Lectura: cada elipse es una "nube" gaussiana; el mixto suma varias nubes.

# Lab sklearn — Checklist (mínimo indispensable)

```
# INPUT:
#   X: shape (N,2)   # datos 2D

# PASOS:
# 1) ajustar GaussianMixture(K)
# 2) leer weights_, means_, covariances_
# 3) predecir componente / probabilidades
# 4) graficar puntos + elipses

# OUTPUT:
#   figura 2D + breve interpretación
```

# Semana 3 — mapa del temario (progreso)

- **Teoría**
- **Lab NumPy**
- **Lab sklearn**
- **Entregable**

# Entregable — qué se evalúa

- **Notebook 1 (NumPy):** condicionamiento por bloques $\rightarrow$ imputación $\rightarrow$ MSE (con baseline).
- **Notebook 2 (sklearn):** GMM 2D $\rightarrow$ parámetros $\rightarrow$ visualización + interpretación breve.
- **Reproducible:** semilla, $N$, % faltantes (si aplica), y una conclusión corta por notebook.

# Semana 4 — Estadística: MLE / ERM

## Likelihood $\to$ log-likelihood $\to$ MLE en Bernoulli/Gaussiana $\to$ ERM y pérdidas sustitutas

Curso de Machine Learning — UAQ (Matemáticas Aplicadas)

Libro base: *Probabilistic Machine Learning: An Introduction* (Kevin P. Murphy)

Semana 4

# Lecturas sugeridas (para estos temas)

- **Capítulo 4 — Statistics**
  - **4.1** Introduction
  - **4.2** Maximum likelihood estimation (MLE)
    - **4.2.1** Definition
    - **4.2.2** Justification for MLE
    - **4.2.3** Bernoulli
    - **4.2.4** Categorical
    - **4.2.5** Univariate Gaussian
    - **4.2.6** Multivariate Gaussian
    - **4.2.7** Linear regression **(vista previa)**
  - **4.3** Empirical risk minimization (ERM)
    - **4.3.1** Minimizing the misclassification rate
    - **4.3.2** Surrogate loss

# Temario de la Semana 4 (qué cubrimos hoy)

- **Teoría (PML)**
  - **1) Lenguaje correcto:** probabilidad vs likelihood **(4.1)**
  - **2) MLE como optimización:** producto $\to$ suma (log) $\to$ NLL **(4.2.1)**
  - **3) Por qué MLE tiene sentido:** "mejor ajuste" dentro del modelo **(4.2.2)**
  - **4) Ejemplos canónicos:** Bernoulli, Categórica, Gaussiana 1D (y MVN) **(4.2.3–4.2.6)**
  - **5) ERM:** riesgo empírico, 0-1 y pérdidas sustitutas **(4.3.1–4.3.2)**
- **Lab NumPy**
  - **Implementar** $\ell(\theta)$, NLL y MLE analítico (Bernoulli + Gaussiana)
  - **Validar** con búsqueda por rejilla (grid) + checks numéricos
- **Lab sklearn**
  - **Equivalencias:** MLE $\leftrightarrow$ minimizar log-loss (intercept-only)
  - **ERM:** ver 0-1 vs log-loss en un barrido de probabilidades
- **Entregable**

# Hilo conductor (dos preguntas "universales")

- **Pregunta A (estadística):** dado un modelo $p(x \mid \theta)$ y datos $D$, ¿qué $\theta$ los hace más plausibles? **(4.2)**
- **Pregunta B (ML):** dado un modelo $f_\theta$ y una pérdida $\ell$, ¿qué $\theta$ minimiza el error promedio en train? **(4.3)**
- **Mensaje:** para modelos probabilísticos, MLE $\equiv$ minimizar NLL, que es un caso de ERM con log-loss.

# Semana 4 — mapa del temario (progreso)

- **Teoría**
- **Lab NumPy**
- **Lab sklearn**
- **Entregable**

# Probabilidad vs likelihood (no es solo semántica)

- **Probabilidad/densidad:** $p(x \mid \theta)$ como función de $x$, con $\theta$ fijo.
- **Likelihood:** $L(\theta; x) = p(x \mid \theta)$ como función de $\theta$, con $x$ fijo. **(4.1)**
- **Error común:** interpretar $L(\theta)$ como una distribución sobre $\theta$ (no tiene por qué sumar/integrar 1).
- Con datos iid: $p(D \mid \theta) = \prod_{i=1}^{n} p(x_i \mid \theta)$.

# Definición de MLE: optimización sobre $\theta$

- **Definición:**

$$\hat{\theta}_{\mathrm{MLE}} = \arg\max_{\theta} p(D \mid \theta). \qquad \textbf{(4.2.1)}$$

- Si los datos son iid:

$$p(D \mid \theta) = \prod_{i=1}^{n} p(x_i \mid \theta).$$

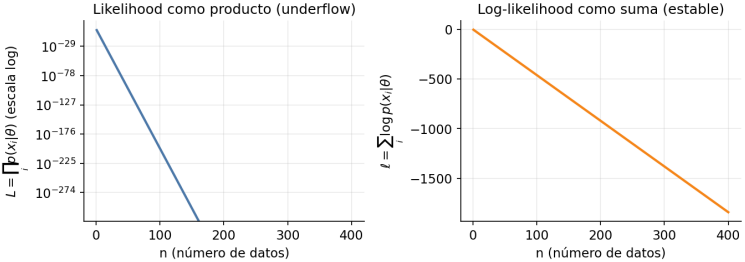
- **Intuición:** elegir el $\theta$ que hace "grandes" las probabilidades/densidades de los datos observados.

# Log-likelihood y NLL: la forma computable

- Producto $\to$ suma:

$$\ell(\theta) = \log p(D \mid \theta) = \sum_{i=1}^{n} \log p(x_i \mid \theta). \qquad \textbf{(4.2.1)}$$

- **NLL (negative log-likelihood):** $\mathrm{NLL}(\theta) = -\ell(\theta)$.
- **Práctico:** evita underflow y es la forma típica de entrenamiento en ML.



# Por qué MLE "tiene sentido" (una lectura útil)

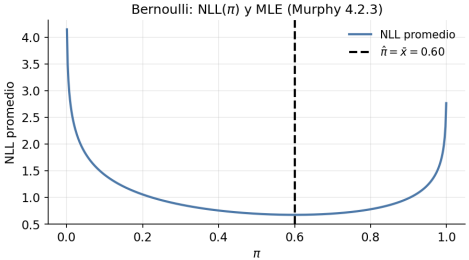
- Maximizar $\ell(\theta)$ equivale a maximizar el promedio $\frac{1}{n}\sum_i \log p(x_i \mid \theta)$. **(4.2.2)**
- **Lectura:** elige el miembro de la familia $p(\cdot \mid \theta)$ que mejor se ajusta a los datos (dentro del modelo).
- **Advertencia:** si el modelo está mal especificado, MLE aun da el "mejor" $\theta$ dentro de esa familia, pero no "arregla" el modelo.

# Ejemplo MLE 1: Bernoulli (resultado = frecuencia)

- Datos $x_i \in \{0,1\}$, $p(x \mid \pi) = \pi^x (1-\pi)^{1-x}$. **(4.2.3)**
- Log-likelihood (sea $S = \sum_i x_i$):

$$\ell(\pi) = S \log \pi + (n - S) \log(1 - \pi).$$

- **MLE:** $\hat{\pi} = \dfrac{S}{n} = \bar{x}$.



# Ejemplo MLE 2: Categórica (resultado = proporciones)

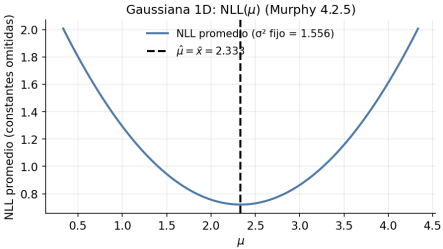
- $x \in \{1, \ldots, K\}$, $p(x = k \mid \boldsymbol{\pi}) = \pi_k$, $\sum_k \pi_k = 1$. **(4.2.4)**
- Con conteos $n_k$: $\ell(\boldsymbol{\pi}) = \sum_{k=1}^{K} n_k \log \pi_k$.
- **MLE:** $\hat{\pi}_k = \dfrac{n_k}{n}$ (frecuencia por clase).
- **Check:** $\sum_k \hat{\pi}_k = 1$. Si $n_k = 0$, evaluar $\log \pi_k$ requiere cuidado numérico.

# Ejemplo MLE 3: Gaussiana 1D (media y varianza MLE)

- $x_i \sim \mathcal{N}(\mu, \sigma^2)$. **(4.2.5)**
- NLL (ignorando constantes):

$$\mathrm{NLL}(\mu, \sigma^2) = \frac{n}{2} \log \sigma^2 + \frac{1}{2\sigma^2} \sum_{i=1}^{n} (x_i - \mu)^2.$$

- **MLE:** $\hat{\mu} = \bar{x}$, $\hat{\sigma}^2 = \frac{1}{n} \sum_i (x_i - \bar{x})^2$.
- Nota: el estimador "insesgado" usa $1/(n-1)$, pero no es el MLE.



# MLE en MVN: misma idea, en $\mathbb{R}^d$

- $\mathbf{x}_i \in \mathbb{R}^d$, $\mathbf{x}_i \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$. **(4.2.6)**
- **MLE:**

$$\hat{\boldsymbol{\mu}} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{x}_i, \qquad \hat{\Sigma} = \frac{1}{n}\sum_{i=1}^{n} (\mathbf{x}_i - \hat{\boldsymbol{\mu}})(\mathbf{x}_i - \hat{\boldsymbol{\mu}})^\top.$$

- **Checks:** $\hat{\Sigma} \succeq 0$; puede ser singular si $n < d$.

# ERM: la versión ML de "ajustar parámetros"

- Riesgo poblacional:

$$R(f) = \mathbb{E}[\ell(f(x), y)].$$

- Riesgo empírico:

$$\hat{R}(f) = \frac{1}{n} \sum_{i=1}^{n} \ell(f(x_i), y_i) \,. \qquad \textbf{(4.3)}$$

- **Mensaje:** en ML casi siempre minimizamos $\hat{R}$ (lo computable).

# Misclassification rate (0-1): por qué es difícil optimizar

- Pérdida 0-1: $\ell_{0/1}(\hat{y}, y) = \mathbb{I}[\hat{y} \neq y]$. **(4.3.1)**
- Empírico: $\hat{R}_{0/1} = \frac{1}{n}\sum_i \mathbb{I}[\hat{y}_i \neq y_i]$ (tasa de error).
- **Problema:** no es suave; pequeños cambios en $\theta$ pueden no cambiar $\hat{y} \Rightarrow$ gradientes poco útiles.

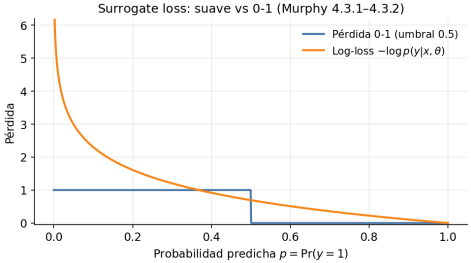
# Surrogate loss: hacer optimizable el problema

- Reemplazamos 0-1 por pérdidas que sean **optimizables** y correlacionen con acertar. **(4.3.2)**
- Para modelos probabilísticos, surrogate natural = **log-loss**:

$$\ell(\theta) = -\log p(y \mid x, \theta).$$

- **Puente:** minimizar NLL es ERM con log-loss:

$$\min_{\theta} \frac{1}{n} \sum_{i} -\log p(y_i \mid x_i, \theta).$$



# Semana 4 — mapa del temario (progreso)

- **Teoría**
- **Lab NumPy**
- **Lab sklearn**
- **Entregable**

# Lab NumPy — objetivo general

- Implementar $\ell(\theta)$ y $\mathrm{NLL}(\theta)$ para Bernoulli y Gaussiana. **(4.2.1)**
- Calcular MLE **analítico** y validarlo con grid search. **(4.2.3, 4.2.5)**
- Incluir checks: clipping, $\sigma^2 > 0$, y comparación de mínimos.

# Lab NumPy — checklist (qué debe aparecer en tu notebook)

- **Bernoulli:** $\hat{\pi} = \bar{x}$, curva $\mathrm{NLL}(\pi)$ por rejilla, y mínimo en $\hat{\pi}$. **(4.2.3)**
- **Gaussiana 1D:** $\hat{\mu} = \bar{x}$, $\hat{\sigma}^2$ (MLE), y validación por rejilla. **(4.2.5)**
- **Numerical hygiene:** evitar $\log 0$, evitar $\sigma^2 \leq 0$, usar NLL promedio para comparar.

# Semana 4 — mapa del temario (progreso)

- **Teoría**
- **Lab NumPy**
- **Lab sklearn**
- **Entregable**

# Lab sklearn — objetivo general

- Ver la equivalencia: MLE $\leftrightarrow$ minimizar log-loss (ERM con surrogate). **(4.3.2)**
- Caso simple: **logística intercept-only** $\Rightarrow$ Bernoulli (probabilidad constante). **(4.2.3)**
- Comparar 0-1 vs log-loss en un barrido de $p \in (0, 1)$. **(4.3.1–4.3.2)**

# Logística intercept-only: la versión "ML" del Bernoulli

- Si el modelo predice una probabilidad constante $p$, el MLE Bernoulli es $\hat{p} = \bar{y}$. **(4.2.3)**
- En logística con solo intercept $b$: $p = \sigma(b) = \frac{1}{1+e^{-b}}$.
- Relación: $b = \log \frac{p}{1-p}$ (logit). Así, $\hat{b} = \log \frac{\bar{y}}{1-\bar{y}}$.
- **Check:** la probabilidad promedio predicha debe ser $\approx \bar{y}$ (si no hay regularización fuerte).

# Semana 4 — mapa del temario (progreso)

- **Teoría**
- **Lab NumPy**
- **Lab sklearn**
- **Entregable**

# Entregable — qué se evalúa

- **Slides:** comprensión conceptual (likelihood/log-likelihood/NLL, MLE, ERM).
- **Notebook NumPy:** implementación de NLL + MLE analítico + validación por grid + checks.
- **Notebook sklearn:** equivalencia log-loss/MLE (intercept-only) + comparación con 0-1.
- **Reproducible:** semilla fija, outputs limpios, y 5–8 líneas de interpretación.

# Cierre: tres ideas para llevarte

1. **MLE** = elegir $\theta$ que maximiza $p(D \mid \theta)$ (mejor "explica" los datos). **(4.2)**
2. Trabajamos casi siempre con **log-likelihood/NLL** por estabilidad y porque suma pérdidas. **(4.2.1)**
3. **ERM** + **surrogates**: entrenar = minimizar un promedio de pérdidas; log-loss conecta directo con MLE. **(4.3)**

# Semana 5 — Decision theory y evaluación de modelos

De probabilidades a decisiones: riesgo esperado, costos asimétricos, umbral $\rightarrow$ matriz de confusión, ROC/PR, AUC $\rightarrow$ validación cruzada

Curso de Machine Learning — UAQ (Matemáticas Aplicadas)

Libro base: *Probabilistic Machine Learning: An Introduction* (Kevin P. Murphy)

Semana 5

# Lecturas sugeridas (para estos temas)

- **Capítulo 5 — Decision Theory**
  - **5.1** Bayesian decision theory
    - **5.1.1** Basics (acciones, estados, pérdida, riesgo)
    - **5.1.2** Classification problems (incluye costos asimétricos y reject)
    - **5.1.3** ROC curves
    - **5.1.4** Precision-recall curves
    - **5.1.5** Regression problems (repaso de pérdidas)
    - **5.1.6** Probabilistic prediction problems (log-loss, Brier)
  - **5.4** Empirical risk minimization (vista desde teoría estadística)
- **Capítulo 4 — Statistics**
  - **4.5.4** Picking the regularizer using a validation set
  - **4.5.5** Cross-validation

# Temario de la Semana 5 (qué cubrimos hoy)

- **Teoría (PML)**
  - **1) Decision theory:** acciones, estados, pérdida, riesgo esperado **(5.1.1)**
  - **2) Clasificación:** pérdida 0-1, regla MAP y matriz de costos **(5.1.2)**
  - **3) Costos asimétricos:** cómo cambian los umbrales de decisión **(5.1.2.2)**
  - **4) Métricas:** matriz de confusión $\rightarrow$ precision/recall, TPR/FPR, specificity **(5.1.3–5.1.4)**
  - **5) Curvas ROC/PR y AUC:** interpretación y efecto del desbalance **(5.1.3–5.1.4)**
  - **6) Validación y CV:** riesgo empírico vs poblacional, K-fold **(1.2.3, 4.5.4–4.5.5, 5.4.3)**
- **Lab NumPy**
  - Construir matrices de confusión, TPR/FPR y precision/recall desde $\{y, s\}$
  - Generar curvas ROC/PR y calcular AUC / PR-AUC manualmente
  - Elegir umbral óptimo en función de costos asimétricos
- **Lab sklearn**
  - Pipeline de evaluación con `StratifiedKFold` (práctica externa)
  - `confusion_matrix`, `roc_curve`, `precision_recall_curve`
  - Reportar ROC-AUC, PR-AUC y métricas por fold en una tabla
- **Entregable**

# Hilo conductor: de “ajustar” a “decidir y evaluar”

- Semana 4: vimos **MLE/ERM** como $\min_\theta \hat{R}(\theta)$ (ajustar parámetros al train).
- Semana 5: suponemos que ya tenemos $p(y \mid x)$ o scores, y ahora toca:
  - **(a) Decidir:** traducir probabilidades en acciones, usando *pérdidas/costos*.
  - **(b) Evaluar:** estimar qué tan bien funcionará el modelo fuera de la muestra.
- **Idea central:** *decision theory* + *evaluación* = puente entre modelos probabilísticos y problemas reales.

# Semana 5 — mapa del temario (progreso)

- **Teoría**
- **Lab NumPy**
- **Lab sklearn**
- **Entregable**

# Decision theory: ingredientes básicos

- **Estados del mundo** $h \in \mathcal{H}$: clase verdadera, valor real, etc.
- **Acciones** $a \in \mathcal{A}$: predecir una clase, un número, dar/no dar un tratamiento, elegir un modelo, etc.
- **Pérdida/costo** $\ell(h, a)$: qué tan "mala" es la acción $a$ si la realidad es $h$. **(5.1.1)**
- **Riesgo posterior** dado evidencia $x$:

$$\rho(a \mid x) = \mathbb{E}_{p(h|x)}[\ell(h, a)].$$

- **Regla de decisión Bayesiana**:

$$\pi^*(x) = \arg\min_{a \in \mathcal{A}} \rho(a \mid x).$$

- **Ejemplo mental**: médico que decide entre "no tratar" o "dar fármaco caro con efectos secundarios" a un paciente que puede (o no) tener COVID-19.

# Decision theory: ejemplo COVID-19 (tabla de pérdidas)

- **Estados** $h \in \mathcal{H}$: combinación de
  - edad (joven vs viejo),
  - estado de enfermedad (COVID-19 vs no COVID-19).
- **Acciones** $a \in \mathcal{A}$: $\{$*Nothing* (no tratar), *Drugs* (dar fármaco)$\}$.
- Usamos *QALYs* (quality-adjusted life years) como unidad de costo/beneficio:
  - joven: 60 QALYs esperados si se salva;
  - viejo: 10 QALYs esperados si se salva;
  - el fármaco "cuesta" 8 QALYs (dolor, efectos secundarios).
- **QALY** (*quality-adjusted life year*) $\approx$ "un año de vida en perfecta salud"; años con peor salud aportan menos de 1 QALY.

| State | Nothing | Drugs |
|---|---|---|
| No COVID-19, young | 0 | 8 |
| COVID-19, young | 60 | 8 |
| No COVID-19, old | 0 | 8 |
| COVID-19, old | 10 | 8 |

# Decision theory: cómo leer la tabla de pérdidas

- La tabla no muestra los QALYs que la persona *tiene*, sino cuántos QALYs **pierde** por la combinación estado/acción.
- Fila "No COVID-19, young":
  - **Nothing = 0**: el joven sano tiene muchos QALYs de vida potencial, pero la decisión de no tratar *no le quita ninguno*.
  - **Drugs = 8**: tratarlo innecesariamente le hace perder 8 QALYs (efectos secundarios, sufrimiento, secuelas).
- Fila "COVID-19, young":
  - **Nothing = 60**: si no se trata y muere, se pierden los 60 QALYs que habría tenido.
  - **Drugs = 8**: el tratamiento le salva la vida pero "cuesta" 8 QALYs de calidad.
- Igual razonamiento para el paciente viejo con COVID-19 (10 vs 8 QALYs de pérdida).

# Riesgo esperado y política óptima (ejemplo COVID-19)

- Dado lo que observamos $x$ (p. ej. resultado del test y edad), el **riesgo** de una acción $a$ es la pérdida esperada:

$$\rho(a \mid x) = \mathbb{E}[\ell(h, a) \mid x] = \sum_{h \in \mathcal{H}} \ell(h, a)\, p(h \mid x).$$

- En el ejemplo, una acción es $a =$ **Nothing** (*no hacer nada*). Llamamos **cost-noop** al riesgo de esa acción:

$$\text{cost-noop}(x) = \rho(\text{Nothing} \mid x).$$

- La **política óptima** elige, para cada $x$, la acción con menor riesgo:

$$\pi^*(x) = \arg\min_{a \in \mathcal{A}} \rho(a \mid x).$$

- Los **inputs** del ejemplo son: `test` (negativo/positivo), age (joven/viejo) y $p(\text{COVID} \mid x)$. En la siguiente slide veremos cómo se calcula cost-noop; al final tendremos la tabla con la decisión óptima.

# Ejemplo COVID-19: cálculo de `cost-noop` (fórmula)

- Por definición, el cost-noop es el riesgo de la acción "no hacer nada":

$$\text{cost-noop}(x) = \mathbb{E}[\ell(h, \text{Nothing}) \mid x] = \sum_h \ell(h, \text{Nothing})\, p(h \mid x).$$

- **Paciente joven**: de la tabla de pérdidas, $\ell(\text{No COVID}, \text{young}, \text{Nothing}) = 0$ y $\ell(\text{COVID}, \text{young}, \text{Nothing}) = 60$. Entonces

$$\text{cost-noop} = 0 \cdot p(\text{No COVID} \mid x) + 60 \cdot p(\text{COVID} \mid x) = 60\, p(\text{COVID} \mid x).$$

- **Paciente mayor**: $\ell(\text{No COVID}, \text{old}, \text{Nothing}) = 0$, $\ell(\text{COVID}, \text{old}, \text{Nothing}) = 10$, luego

$$\text{cost-noop} = 10\, p(\text{COVID} \mid x).$$

- Los valores de la tabla se obtienen sustituyendo $p(\text{COVID} \mid x)$ por las probabilidades que da el test (siguiente slide).

# Ejemplo COVID-19: `cost-noop` (valores numéricos)

- Con las probabilidades del test: $p(\text{COVID} \mid \text{test} = 0) \approx 0{,}014$, $p(\text{COVID} \mid \text{test} = 1) \approx 0{,}7955$.
- Sustituyendo en las fórmulas anteriores:
  - **Test negativo**: joven $60 \times 0{,}014 = 0{,}84$; mayor $10 \times 0{,}014 = 0{,}14$.
  - **Test positivo**: joven $60 \times 0{,}7955 = 47{,}73$; mayor $10 \times 0{,}7955 = 7{,}95$.

| test | age | pr(covid) | cost-noop | cost-drugs | action |
|---|---|---|---|---|---|
| 0 | 0 | 0.01 | 0.84 | 8.00 | 0 |
| 0 | 1 | 0.01 | 0.14 | 8.00 | 0 |
| 1 | 0 | 0.80 | 47.73 | 8.00 | 1 |
| 1 | 1 | 0.80 | 7.95 | 8.00 | 0 |

- **Comparación**: cost-drugs $= 8$ siempre. Si cost-noop $< 8$ conviene no tratar (action=0); si cost-noop $> 8$, tratar (action=1). Por eso, con test positivo y paciente joven, la política óptima es tratar; con test positivo y mayor, no tratar (7.95 y 8 son muy cercanos).

# De pérdida a utilidad: el mismo principio

- En muchos textos se usa **utilidad** $U(h, a)$ en vez de pérdida $\ell(h, a)$.
- Relación simple: $U(h, a) = -\ell(h, a)$.
- **Máxima utilidad esperada**:

$$\pi^*(x) = \arg\max_{a \in \mathcal{A}} \mathbb{E}_{p(h|x)}[U(h, a)].$$

- En ML solemos pensar en **minimizar pérdida promedio** (riesgo) $\leftrightarrow$ **maximizar utilidad esperada**.

# Clasificación binaria con pérdida 0-1

- Estados y acciones son las clases: $\mathcal{H} = \mathcal{A} = \{0, 1\}$. **(5.1.2.1)**
- **Pérdida** 0-1:

$$\ell_{0/1}(y^*, \hat{y}) = \mathbf{1}[y^* \neq \hat{y}].$$

- Riesgo posterior de predecir $\hat{y}$:

$$\rho(\hat{y} \mid x) = 1 - p(y^* = \hat{y} \mid x).$$

- **Regla óptima**: elegir la clase más probable (MAP):

$$\hat{y}(x) = \arg \max_{y \in \{0,1\}} p(y \mid x).$$

# Costos asimétricos en binario: matriz de pérdida

- En muchos problemas, **no todos los errores cuestan igual**.
- Matriz de pérdida general:

$$\ell(y^*, \hat{y}) = \begin{pmatrix} \ell_{00} & \ell_{01} \\ \ell_{10} & \ell_{11} \end{pmatrix},$$

donde filas = verdad, columnas = predicción. **(5.1.2.2)**

- Ejemplo clásico: detección de enfermedad
  - FP (decir "enfermo" cuando está sano) = costo moderado.
  - FN (decir "sano" cuando está enfermo) = costo muy alto.

# Costos asimétricos $\Rightarrow$ umbral distinto de 0.5

- Sea $p_1 = p(y^* = 1 \mid x)$, $p_0 = 1 - p_1$, donde $y^*$ es la **etiqueta verdadera** (ground truth) y $\hat{y}$ la predicción.
- Así, $p_1 =$ probabilidad de que la clase real sea 1 dado $x$; $p_0 =$ probabilidad de que sea 0.
- Riesgo de predecir $\hat{y} = 1$:
$$\rho(1 \mid x) = \ell_{01} p_0 + \ell_{11} p_1.$$
- Riesgo de predecir $\hat{y} = 0$:
$$\rho(0 \mid x) = \ell_{00} p_0 + \ell_{10} p_1.$$
- Caso típico $\ell_{00} = \ell_{11} = 0$: predecir 1 si
$$p_1 \geq \frac{\ell_{01}}{\ell_{01} + \ell_{10}}.$$
- Si un FN cuesta $c$ veces más que un FP ($\ell_{10} = c\,\ell_{01}$):
$$p_1 \geq \frac{1}{1 + c}.$$
- **Ejemplo**: $c = 4 \Rightarrow$ umbral $1/5 = 0{,}2$ (somos mucho más "sensibles").

# Costos asimétricos: de dónde sale el umbral

- Con $\ell_{00} = \ell_{11} = 0$ (acertar no cuesta), el riesgo de predecir 1 es $\rho(1 \mid x) = \ell_{01}(1 - p_1)$; el de predecir 0 es $\rho(0 \mid x) = \ell_{10} p_1$.
- Predecimos 1 cuando $\rho(1 \mid x) \leq \rho(0 \mid x)$, es decir

$$\ell_{01}(1 - p_1) \leq \ell_{10} p_1 \iff \ell_{01} - \ell_{01} p_1 \leq \ell_{10} p_1 \iff \ell_{01} \leq p_1(\ell_{01} + \ell_{10}) \iff p_1 \geq \frac{\ell_{01}}{\ell_{01} + \ell_{10}}.$$

- Si $\ell_{10} = c\,\ell_{01}$ (un FN cuesta $c$ veces un FP):

$$p_1 \geq \frac{1}{1 + c}.$$

- **Interpretación**: $c = 4 \Rightarrow$ umbral 0.2. Bajamos el umbral respecto a 0.5 para ser más sensibles: preferimos más FP que perder casos reales (FN).

# Costos asimétricos: umbral 0.5 vs umbral óptimo

- Con **umbral 0.5** (costos simétricos): predecimos 1 solo si $p_1 \geq 0{,}5$.
- Con **umbral** $1/(1+c)$ (FN cuesta $c$ veces un FP): predecimos 1 si $p_1 \geq 1/(1+c)$.
- Si $c > 1$, entonces $1/(1+c) < 0{,}5$: el umbral óptimo es *menor* que 0.5.
- **Ejemplo** $c = 4$: umbral 0,2. Un modelo que da $p_1 = 0{,}3$:
  - Con 0.5: predeciríamos 0 ("sano"); con 0.2: predecimos 1 ("tratar").
  - Priorizamos no perder enfermos (menos FN), aunque haya más falsas alarmas (más FP).

# Costos asimétricos: ejemplo numérico completo

- Supongamos la siguiente matriz de pérdida para un clasificador médico:

$$\ell(y^*, \hat{y}) = \begin{pmatrix} 0 & 1 \\ 5 & 0 \end{pmatrix},$$

  donde $y^* = 0 =$ sano, $y^* = 1 =$ enfermo.
- Entonces un FN (enfermo etiquetado como sano) cuesta 5 veces un FP $\Rightarrow c = 5$.
- El **umbral óptimo** según decision theory es

$$p_1^\star = \frac{1}{1+c} = \frac{1}{6} \approx 0{,}17.$$

- **Interpretación práctica**:
  - Si $p(y = 1 \mid x) = 0{,}20$: bajo 0,5 dirías "sano"; bajo 0,17 ya vale la pena tratar.
  - Si $p(y = 1 \mid x) = 0{,}05$: incluso con costos asimétricos, sigue por debajo del umbral 0,17.

# Matriz de confusión: conectar decisiones con conteos

- Para un clasificador binario con umbral fijo:
  - **TP**: verdaderos positivos
  - **FP**: falsos positivos
  - **TN**: verdaderos negativos
  - **FN**: falsos negativos
- Tabla (forma de **Tabla 5.3**):

| | $\hat{y} = 0$ | $\hat{y} = 1$ |
|---|---|---|
| $y = 0$ | $TN$ | $FP$ |
| $y = 1$ | $FN$ | $TP$ |

# Derivar métricas desde la matriz de confusión

- Normalizando por filas (condicionado en la verdad): **(5.1.3.1, 5.1.3.2)**
  - **TPR / recall / sensitivity**: $\mathrm{TPR} = \dfrac{TP}{TP + FN}$.
  - **TNR / specificity**: $\mathrm{TNR} = \dfrac{TN}{TN + FP}$.
  - **FPR**: $\mathrm{FPR} = 1 - \mathrm{TNR} = \dfrac{FP}{FP + TN}$.
- Normalizando por columnas (condicionado en la predicción):
  - **Precision / PPV**: $\mathrm{Prec} = \dfrac{TP}{TP + FP}$.
  - **NPV**: $\mathrm{NPV} = \dfrac{TN}{TN + FN}$.
- **Idea**: todas las métricas que usamos (accuracy, recall, precision, etc.) son distintas "vistas" de la misma tabla.

# Matriz de confusión: ejemplo numérico

| | $\hat{y} = 0$ | $\hat{y} = 1$ |
|---|---|---|
| $y = 0$ | 50 | 10 |
| $y = 1$ | 5 | 35 |

- Total negativos verdaderos $N = 60$, positivos verdaderos $P = 40$.
- Accuracy $= (50 + 35)/100 = 0{,}85$.

- **Recall / TPR**:

$$\mathrm{TPR} = \frac{TP}{TP + FN} = \frac{35}{35 + 5} = 0{,}875.$$

- **FPR**:

$$\mathrm{FPR} = \frac{FP}{FP + TN} = \frac{10}{10 + 50} \approx 0{,}167.$$

- **Precision**:

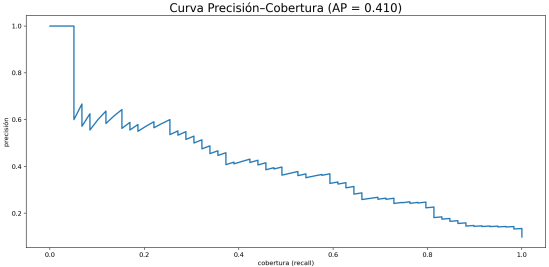
$$\mathrm{Prec} = \frac{TP}{TP + FP} = \frac{35}{35 + 10} \approx 0{,}78.$$

- **NPV**:

$$\mathrm{NPV} = \frac{TN}{TN + FN} = \frac{50}{50 + 5} \approx 0{,}91.$$

# Curva ROC: barrer el umbral

- Si el modelo produce un **score** $s(x)$ (p. ej. $p(y = 1 \mid x)$), definimos $\hat{y}_\tau(x) = \mathbf{1}[s(x) \geq \tau]$.
- Para cada $\tau$ calculamos $\mathrm{TPR}_\tau$ y $\mathrm{FPR}_\tau$. **(5.1.3)**
- La **curva ROC** grafica $\mathrm{TPR}_\tau$ vs $\mathrm{FPR}_\tau$ al variar $\tau \in [0, 1]$.
- **Extremos**:
  - $\tau \to 0$: todo es positivo ($\mathrm{TPR} \approx 1$, $\mathrm{FPR} \approx 1$).
  - $\tau \to 1$: nadie es positivo ($\mathrm{TPR} \approx 0$, $\mathrm{FPR} \approx 0$).



# AUC, EER y lectura rápida de ROC

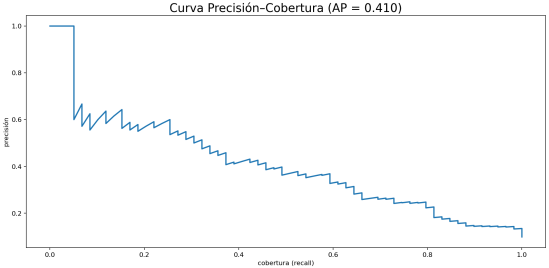
- **AUC** (Area Under the ROC Curve):
  - $\mathrm{AUC} = 1$: clasificador perfecto (curva pegada a la esquina superior izquierda).
  - $\mathrm{AUC} \approx 0{,}5$: como lanzar una moneda (diagonal).
- **Equal Error Rate (EER)**:
  - Punto donde $\mathrm{FPR} = \mathrm{FNR} = 1 - \mathrm{TPR}$. **(5.1.3.2)**
  - Se visualiza como intersección de la ROC con la diagonal descendente.
- **Uso práctico**: ROC es útil cuando el costo de FP/FN no está fijado, y queremos comparar modelos "globalmente".

# Curva Precision–Recall (PR): cuando los positivos son raros

- Recordemos:

$$\mathrm{Prec}_\tau = \frac{TP_\tau}{TP_\tau + FP_\tau}, \qquad \mathrm{Rec}_\tau = \frac{TP_\tau}{TP_\tau + FN_\tau}. \qquad \textbf{(5.1.4.1)}$$

- La **curva PR** grafica $\mathrm{Prec}_\tau$ vs $\mathrm{Rec}_\tau$ al variar $\tau$.
- Es especialmente informativa cuando hay **fuerte desbalance** ($P \ll N$), donde ROC puede verse "demasiado bonita".
- **PR-AUC / AP**: área (o average precision) bajo la curva PR. **(5.1.4.2)**



# Mini-ejemplo ROC/PR con 5 ejemplos

- Dataset pequeño (ordenado por score descendente):

| $n$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $y_n$ | 1 | 0 | 1 | 0 | 1 |
| $s_n$ | 0,9 | 0,8 | 0,6 | 0,4 | 0,3 |

- Hay $P = 3$ positivos, $N = 2$ negativos.
- Si usamos $\tau = 0{,}75$:
  - $\hat{y} = (1, 1, 0, 0, 0) \Rightarrow TP = 1, FP = 1, TN = 1, FN = 2.$
  - $\mathrm{TPR} = 1/3$, $\mathrm{FPR} = 1/2$, $\mathrm{Prec} = 1/2$, $\mathrm{Rec} = 1/3$.
- Si usamos $\tau = 0{,}5$:
  - $\hat{y} = (1, 1, 1, 0, 0) \Rightarrow TP = 2, FP = 1, TN = 1, FN = 1.$
  - $\mathrm{TPR} = 2/3$, $\mathrm{FPR} = 1/2$, $\mathrm{Prec} = 2/3$, $\mathrm{Rec} = 2/3$.

# Mini-ejemplo ROC/PR (cont.): puntos en las curvas

- Para el dataset anterior, podemos armar una pequeña tabla:

| $\tau$ | TPR | FPR | Prec | Rec |
|---|---|---|---|---|
| 1,0 | 0 | 0 | – | – |
| 0,75 | 1/3 | 1/2 | 1/2 | 1/3 |
| 0,5 | 2/3 | 1/2 | 2/3 | 2/3 |
| 0,25 | 1 | 1 | 3/5 | 1 |

- **ROC**: puntos $(\mathrm{FPR}, \mathrm{TPR}) = (0,0), (0{,}5, 1/3), (0{,}5, 2/3), (1,1)$.
- **PR**: puntos $(\mathrm{Rec}, \mathrm{Prec}) = (0,-), (1/3, 1/2), (2/3, 2/3), (1, 3/5)$.
- Con estos pocos puntos ya se puede:
  - visualizar cómo el **umbral** barre las curvas;
  - aproximar AUC/PR-AUC con la regla del trapecio (como en el Lab NumPy).

# F-scores: combinar precisión y recall en un solo número

- Para un umbral fijo $(\mathrm{Prec}, \mathrm{Rec})$, el $\mathbf{F}_\beta$ se define como: **(5.1.4.3)**

$$F_\beta = \frac{(1+\beta^2)\,\mathrm{Prec}\,\mathrm{Rec}}{\beta^2\,\mathrm{Prec} + \mathrm{Rec}}.$$

- Casos especiales:
  - $F_1$: media armónica de precisión y recall (pesa ambos por igual).
  - $\beta > 1$: da más peso a *recall* (importa más cubrir positivos).
  - $\beta < 1$: da más peso a *precision* (importa más evitar FP).
- **Interpretación**: útil para seleccionar el umbral cuando tenemos una preferencia clara por precision o recall.

# Desbalance de clases: ROC vs PR

- ROC usa tasas $\mathrm{TPR}$ y $\mathrm{FPR}$ que **no cambian** si escalamos P y N manteniendo la proporción. **(5.1.3.3)**
- En problemas muy desbalanceados, ROC puede verse buena aunque haya muchos FP absolutos.
- PR (y F-score) sí son **sensibles** a la prevalencia:

$$\mathrm{Prec} = \frac{\mathrm{TPR}}{\mathrm{TPR} + \frac{1}{r}\mathrm{FPR}}, \quad r = \frac{P}{N}. \qquad \textbf{(5.1.4.4)}$$

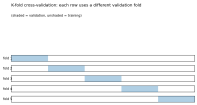
- **Regla práctica**:
  - Clases balanceadas $\rightarrow$ ROC-AUC es razonable.
  - Clases raras $\rightarrow$ mira PR-AUC / AP y F-scores.

# Validación: train / validation / test y leakage

- Separar los datos en:
  - **Train**: ajustar parámetros $\theta$.
  - **Validation**: escoger hiperparámetros (p. ej. $\lambda$, arquitectura, umbral).
  - **Test**: estimar $L(\theta^*; p^*)$ una sola vez. **(1.2.3, 4.5.4)**
- **Leakage**: usar test (directa o indirectamente) para tomar decisiones de modelo/hiperparámetros.
- **Consecuencia**: subestimar el error real; el modelo parece mejor de lo que realmente es.

# Cross-validation (CV): usar mejor los datos

- Problema: con pocos datos, separar un validation set fijo puede ser caro.
- **K-fold CV** ($K$-pliegues): **(4.5.5, 5.4.3)**
  - Partimos train en $K$ folds $D_1, \ldots, D_K$.
  - Para cada $k$: entrenamos en $D_{-k}$ y evaluamos en $D_k$.
  - Promediamos la métrica (MSE, AUC, F1, etc.) sobre los $K$ folds.
- **Uso típico**: escoger hiperparámetros (p. ej. $\lambda$ de regularización) y comparar modelos.



# Cross-validation (CV): ejemplo numérico sencillo

- Supongamos $K = 3$ folds y medimos **accuracy** para un modelo:

| Fold | Accuracy |
|---|---|
| 1 | 0,82 |
| 2 | 0,79 |
| 3 | 0,87 |

- **Promedio**: $\bar{a} = (0{,}82 + 0{,}79 + 0{,}87)/3 \approx 0{,}83$.
- **Desviación estándar** (muestral):

$$s \approx \sqrt{\frac{(0{,}82 - \bar{a})^2 + (0{,}79 - \bar{a})^2 + (0{,}87 - \bar{a})^2}{3 - 1}} \approx 0{,}04.$$

- Reporte típico: accuracy CV $\approx 0{,}83 \pm 0{,}04$.
- **Lectura**: el modelo parece razonablemente estable, pero hay variabilidad entre folds (dependiente del split).

# Semana 5 — mapa del temario (progreso)

- **Teoría**
- **Lab NumPy**
- **Lab sklearn**
- **Entregable**

# Lab NumPy — objetivo general

- Tomar $\{y_n \in \{0,1\}, s_n \in [0,1]\}$ y construir:
  - Matrices de confusión para distintos umbrales $\tau$.
  - Curvas ROC y PR, con AUC / PR-AUC vía regla del trapecio.
- Implementar una función de **riesgo empírico** con matriz de costos $\ell_{ij}$ y buscar el **umbral óptimo**.

# Lab NumPy — checklist (qué debe aparecer en tu notebook)

- **Funciones básicas**:
  - `confusion_counts(y_true, scores, tau)` $\rightarrow$ TP, FP, TN, FN.
  - `metrics_from_confusion(...)` $\rightarrow$ accuracy, TPR, FPR, precision, recall, F1.
- **Curvas y áreas**:
  - `roc_curve_manual` y `pr_curve_manual`.
  - `auc_trapezoidal(x, y)` para ROC-AUC y PR-AUC.
  - Gráficos ROC y PR etiquetados (similar a las figuras del libro).
- **Costos asimétricos**:
  - Función `empirical_risk_tau(y, scores, tau, cost_matrix)`.
  - Gráfico de $\hat{L}(\tau)$ vs $\tau$ para distintos cocientes de costos FP/FN.
  - Comparar el $\tau$ que minimiza riesgo con el umbral $1/(1+c)$ teórico.

# Semana 5 — mapa del temario (progreso)

- **Teoría**
- **Lab NumPy**
- **Lab sklearn**
- **Entregable**

# Lab sklearn — objetivo general

- Construir un pipeline mínimo de evaluación para clasificación binaria:
  - `StratifiedKFold` para estimar métricas con CV (nota práctica: no aparece como tal en el libro).
  - `confusion_matrix`, `roc_curve`, `roc_auc_score`.
  - `precision_recall_curve`, `average_precision_score`.
- Reportar métricas por fold y su promedio $\pm$ desviación estándar.

# Lab sklearn — checklist (estructura sugerida)

- **Datos y modelo**:
  - Dataset binario con cierto desbalance.
  - Modelo sencillo (p. ej. `LogisticRegression` o `RandomForestClassifier`).
- **Cross-validation**:
  - `StratifiedKFold(n_splits=K, shuffle=True, random_state=...)`.
  - Para cada fold: entrenar, obtener `predict_proba` y `predict`.
- **Métricas por fold**:
  - Accuracy, ROC-AUC, PR-AUC (AP), F1.
  - Matriz de confusión.
- **Tabla final**:
  - Filas = folds, columnas = métricas.
  - Fila extra con promedio y desviación estándar.

# Semana 5 — mapa del temario (progreso)

- **Teoría**
- **Lab NumPy**
- **Lab sklearn**
- **Entregable**

# Entregable — qué se evalúa

- **Slides:** claridad en:
  - definición de pérdida, riesgo esperado y reglas de decisión;
  - relación entre matriz de confusión, ROC/PR y métricas derivadas;
  - papel de train/validation/test y CV.
- **Notebook NumPy:** construcción manual y correcta de métricas/curvas + selección de umbral con costos.
- **Notebook sklearn:** pipeline de evaluación con CV + tabla de métricas bien interpretada.

# Cierre: tres ideas para llevarte

1. **Decision theory** formaliza "qué acción tomar" como arg mín de riesgo esperado (pérdida + probabilidades). **(5.1.1–5.1.2)**
2. **Costos asimétricos** cambian el **umbral óptimo** y, por tanto, la región de operación en la ROC/PR. **(5.1.2.2, 5.1.3–5.1.4)**
3. **Evaluar bien** (train/val/test, CV) es clave para estimar el **riesgo poblacional** y no sobreestimar el modelo. **(1.2.3, 4.5.5, 5.4.3)**

# Semana 6 — Information Theory para Machine Learning

## Entropía, cross-entropy, KL divergence y su papel como loss

Curso de Machine Learning — UAQ (Matemáticas Aplicadas)

Libro base: *Probabilistic Machine Learning: An Introduction* (Kevin P. Murphy)

Semana 6

# Lecturas sugeridas (para estos temas)

- **Capítulo 6 — Information theory**
  - **6.1** Entropy
    - **6.1.1** Entropy for discrete random variables
    - **6.1.2** Cross entropy
    - **6.1.3–6.1.5** Joint / conditional entropy, perplexity
  - **6.2** Relative entropy (KL divergence)
    - **6.2.1–6.2.6** Definición, interpretación, no-negatividad, KL y MLE

# Temario de la Semana 6 (qué cubrimos hoy)

- **Teoría (PML)**
  - **1) Surprisal:** $I(x) = -\log p(x)$, unidades (bits/nats) **(6.1.1)**
  - **2) Entropía:** $H(X)$ como incertidumbre promedio; ejemplos discretos **(6.1.1)**
  - **3) Cross-entropy:** $H(p, q)$, interpretación codificación / modelo **(6.1.2)**
  - **4) KL:** $\mathrm{KL}(p\|q)$, no-negatividad, orden $p$ vs $q$ **(6.2)**
  - **5) Identidad:** $H(p, q) = H(p) + \mathrm{KL}(p\|q)$ **(6.1.2, 6.2)**
  - **6) ML:** CE/NLL en etiquetas one-hot, enlace con MLE e incertidumbre del modelo **(6.1.2, 6.2)**
- **Lab NumPy**
  - Calcular $H(p)$, $H(p, q)$ y $\mathrm{KL}(p\|q)$ a mano en ejemplos pequeños
  - Curva de entropía Bernoulli $H(\theta)$; verificar casos límite numéricamente
- **Lab sklearn**
  - `log_loss` y equivalencia con CE promedio en clasificación
  - Comparar CE manual vs implementación en un mini-dataset
- **Entregable**

# Hilo conductor: de “evaluar aciertos” a “evaluar probabilidades”

- Semana 5: métricas de clasificación y curvas ROC/PR miden, sobre todo, **decisiones** y **aciertos** bajo umbrales.
- Semana 6: si el modelo entrega una **distribución** $\hat{p}(y \mid x)$, necesitamos una loss coherente con probabilidades.
- **Idea central:** surprisal $\rightarrow$ entropía $\rightarrow$ cross-entropy $\rightarrow$ KL conectan incertidumbre, codificación y entrenamiento (MLE $\equiv$ minimizar NLL/CE).

# Semana 6 — mapa del temario (progreso)

- **Teoría**
- **Lab NumPy**
- **Lab sklearn**
- **Entregable**

# ¿Qué estudia information theory?

- **Pregunta central:** ¿cuánta incertidumbre hay en una fuente aleatoria y cuánto cuesta describir sus resultados?
- **Shannon (1948):** propone medidas cuantitativas para:
  - la **incertidumbre** de una variable aleatoria,
  - la **sorpresa** de observar un evento,
  - el costo de usar un **modelo incorrecto** para describir los datos.
- **Idea guía de la semana:** pasaremos de la sorpresa de un evento individual a la pérdida promedio de un modelo probabilístico.
- Esa ruta conceptual será:

$$\text{surprisal} \longrightarrow \text{entropía} \longrightarrow \text{cross-entropy} \longrightarrow \mathrm{KL}.$$

# ¿Por qué importa en machine learning?

- En muchos modelos de ML no solo queremos **predecir una clase**, sino asignar una **distribución de probabilidad** sobre las clases.
- Esto requiere una forma coherente de:
  - medir la **incertidumbre** del modelo,
  - **comparar** distribuciones (datos vs modelo),
  - definir **funciones de pérdida** basadas en probabilidad.
- Las cantidades de information theory (entropía, cross-entropy, divergencia KL) serán los ladrillos para:
  - diseñar la loss estándar de clasificación,
  - entender por qué maximizar likelihood equivale a minimizar NLL/CE,
  - preparar métodos probabilísticos más avanzados.

# Self-information / surprisal: definición e intuición

- Sea un evento $x$ con probabilidad $p(x) \in (0, 1]$.
- **Definición (Shannon):**

$$I(x) \triangleq -\log p(x)$$

- **Unidades:**
  - base 2 $\Rightarrow$ bits,
  - base $e$ $\Rightarrow$ nats.
- **Intuición:**
  - cuanto más improbable es $x$, mayor es su sorpresa/información;
  - si $p(x) = 1$ (evento seguro), entonces $I(x) = 0$: no hay sorpresa.

# Ejemplo numérico de surprisal

- Evento A: lanzar una moneda justa y obtener cara.

$$p(A) = 0{,}5, \quad I(A) = -\log_2(0{,}5) = 1 \text{ bit}$$

- Evento B: lanzar un dado justo y obtener un 6.

$$p(B) = \tfrac{1}{6}, \quad I(B) = -\log_2\left(\tfrac{1}{6}\right) \approx 2{,}585 \text{ bits}$$

- **Lectura:** observar un 6 específico en un dado justo aporta más información que observar cara en una moneda justa.

**Nota:** Confundir "información" con "importancia" del evento. Un evento raro tiene mucha información en el sentido de sorpresa, no necesariamente en el sentido de relevancia práctica.

# Analogía útil para explicar surprisal

- Imagina un sistema de **alertas**:
  - "Todo funciona normal" (muy frecuente) $\rightarrow$ poca sorpresa.
  - "El servidor crítico está caído" (muy raro) $\rightarrow$ mucha sorpresa.
- El valor $I(x)$ mide "**cuán inesperado**" fue el evento.
- En ML, si el modelo asigna probabilidad muy baja a la clase verdadera, la loss será alta: el modelo se "sorprende" demasiado por lo que pasó en realidad.

| | Evento frecuente | Evento raro | Evento muy raro |
|---|---|---|---|
| Probabilidad $p(x)$ | 0.8 | 0.15 | 0.05 |
| Sorpresa $I(x) = -\log_2 p(x)$ | $\approx 0{,}32$ | $\approx 2{,}74$ | $\approx 4{,}32$ |

# Entropía: definición formal

- Sea $X$ una variable discreta con distribución $p(x)$ sobre un conjunto finito $\mathcal{X}$.
- **Definición de entropía de Shannon:**

$$H(X) \triangleq -\sum_{x \in \mathcal{X}} p(x) \log p(x)$$

- **Interpretaciones equivalentes:**
  - incertidumbre promedio de $X$,
  - información esperada de los eventos $x$ (promedio de $I(x)$),
  - longitud promedio mínima (en bits) de un código óptimo para $X$.
- **Notación alternativa:** $H(p)$ cuando queremos enfatizar que depende de la distribución $p$.

# Entropía como esperanza de surprisal

- Ya definimos la sorpresa de un evento individual:

$$I(x) = -\log p(x).$$

- Si ahora $X \sim p$, la sorpresa **promedio** es:

$$\mathbb{E}_p[I(X)] = \sum_x p(x)\, I(x).$$

- Sustituyendo $I(x) = -\log p(x)$:

$$\mathbb{E}_p[I(X)] = \sum_x p(x)\,[-\log p(x)] = -\sum_x p(x) \log p(x).$$

- **Conclusión:**

$$H(X) = \mathbb{E}_p[I(X)].$$

- Esta identidad conecta de forma natural evento individual $\rightarrow$ incertidumbre promedio.

# Ejemplo: entropía de una distribución discreta simple

- Supón $X \in \{a, b, c\}$ con:

$$p(a) = 0{,}5, \quad p(b) = 0{,}25, \quad p(c) = 0{,}25.$$

- Calculamos:

$$H(X) = -\big[0{,}5 \log_2 0{,}5 + 0{,}25 \log_2 0{,}25 + 0{,}25 \log_2 0{,}25\big].$$

- Paso a paso:

$$\log_2 0{,}5 = -1, \quad \log_2 0{,}25 = -2$$

$$H(X) = -\big[0{,}5(-1) + 0{,}25(-2) + 0{,}25(-2)\big] = -\big[-0{,}5 - 0{,}5 - 0{,}5\big] = 1{,}5 \text{ bits}.$$

- **Lectura:** en promedio, "necesitamos" 1.5 bits para codificar $X$ de forma óptima.

# Entropía y dispersión de la distribución

- Para un alfabeto fijo:
  - la **entropía es máxima** cuando la distribución es uniforme (toda la probabilidad repartida por igual);
  - la **entropía es mínima** (0) cuando la distribución es degenerada (toda la masa en un solo punto).
- Intuición: cuanta más "dispersa" esté la probabilidad entre muchos valores plausibles, más incertidumbre tenemos; cuando casi todo el peso está en un único valor, apenas dudamos.
- **Mensaje para ML:** una predicción con probabilidades muy repartidas tiene mayor entropía que una predicción muy concentrada en una sola clase.

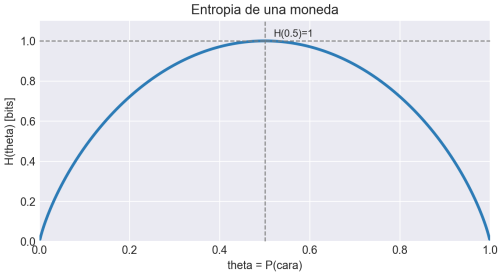
# Entropía de una moneda: visualización

- Para una moneda con parámetro $\theta = \Pr(\text{cara})$, la entropía

$$H(\theta) = -\theta \log_2 \theta - (1-\theta) \log_2 (1-\theta)$$

  mide cuánta incertidumbre hay antes de observar el resultado del lanzamiento.
- Analíticamente sabemos que:
  - la incertidumbre es **máxima** en $\theta = 0{,}5$ (moneda justa),
  - tiende a 0 cuando $\theta \to 0$ o $\theta \to 1$ (moneda casi determinista).
- En la siguiente gráfica se ve claramente esta forma "en campana invertida" de $H(\theta)$.

# Entropía de una moneda: curva $H(\theta)$




La entropía es máxima para $\theta = 0{,}5$ (moneda justa) y se aproxima a 0 cuando $\theta$ se acerca a 0 o 1, es decir, cuando la moneda es prácticamente determinista.

# Cross-entropy: definición e interpretación

- Tenemos una **distribución verdadera** $p$ y un **modelo** $q$.
- **Definición:**

$$H(p, q) \triangleq -\sum_x p(x) \log q(x).$$

- **Lectura:**
  - sorpresa promedio de datos generados por $p$, pero evaluada con el modelo $q$,
  - costo promedio de codificar datos de $p$ usando el código óptimo para $q$.
- Si $q = p$, entonces $H(p, q) = H(p)$: el mejor caso es usar el modelo verdadero.

# Ejemplo: cross-entropy entre dos distribuciones sobre $\{a, b\}$

- Verdadera $p$: $\quad p(a) = 0{,}8,\ p(b) = 0{,}2.$
- Modelo $q$: $\quad q(a) = 0{,}6,\ q(b) = 0{,}4.$
- Calculamos:

$$H(p, q) = -\big[0{,}8 \log_2 0{,}6 + 0{,}2 \log_2 0{,}4\big].$$

- Números aproximados:

$$\log_2 0{,}6 \approx -0{,}737, \quad \log_2 0{,}4 \approx -1{,}322,$$

$$H(p, q) \approx -\big[0{,}8(-0{,}737) + 0{,}2(-1{,}322)\big] \approx -\big[-0{,}5896 - 0{,}2644\big] \approx 0{,}854 \text{ bits}.$$

- Como veremos enseguida, este valor es mayor que $H(p)$ por una cantidad exactamente igual a la divergencia KL.

# KL divergence: definición y significado

- **Definición:**

$$\mathrm{KL}(p\|q) \triangleq \sum_x p(x) \log \frac{p(x)}{q(x)}.$$

- **Interpretaciones:**
  - "**penalización** media por usar $q$ en lugar de $p$",
  - exceso de longitud promedio (en bits) al codificar datos de $p$ con un código óptimo para $q$,
  - medida de cuán distinta es $q$ de $p$ (aunque *no* es una distancia simétrica).
- Propiedades clave:
  - $\mathrm{KL}(p\|q) \geq 0$ (**desigualdad de Gibbs**),
  - $\mathrm{KL}(p\|q) = 0 \iff p = q$ (casi en todas partes),
  - en general $\mathrm{KL}(p\|q) \neq \mathrm{KL}(q\|p)$.

# Ejemplo numérico de KL divergence

- Reutilizamos el ejemplo anterior:

$$p(a) = 0{,}8,\ p(b) = 0{,}2, \quad q(a) = 0{,}6,\ q(b) = 0{,}4.$$

- Calculamos:

$$\mathrm{KL}(p\|q) = 0{,}8 \log_2 \frac{0{,}8}{0{,}6} + 0{,}2 \log_2 \frac{0{,}2}{0{,}4}.$$

$$\log_2 \frac{0{,}8}{0{,}6} \approx \log_2(1{,}333) \approx 0{,}415, \quad \log_2 \frac{0{,}2}{0{,}4} = \log_2(0{,}5) = -1.$$

$$\mathrm{KL}(p\|q) \approx 0{,}8(0{,}415) + 0{,}2(-1) \approx 0{,}332 - 0{,}2 = 0{,}132 \text{ bits}.$$

- **Lectura:** al codificar datos de $p$ con un código óptimo para $q$, pagamos en promedio 0.132 bits extra por símbolo.

# Identidad clave: $H(p,q) = H(p) + \mathrm{KL}(p\|q)$

- Recordemos:

$$H(p) = -\sum_x p(x)\log p(x), \quad H(p,q) = -\sum_x p(x)\log q(x).$$

- Queremos mostrar:

$$H(p,q) = H(p) + \mathrm{KL}(p\|q).$$

**Derivación paso a paso:**

$$\mathrm{KL}(p\|q) = \sum_x p(x)\log\frac{p(x)}{q(x)} = \sum_x p(x)\big[\log p(x) - \log q(x)\big].$$

$$\mathrm{KL}(p\|q) = \sum_x p(x)\log p(x) - \sum_x p(x)\log q(x).$$

$$-\sum_x p(x)\log q(x) = -\sum_x p(x)\log p(x) + \mathrm{KL}(p\|q).$$

$$\Rightarrow \quad H(p,q) = H(p) + \mathrm{KL}(p\|q).$$

# Interpretación de la identidad $H(p,q) = H(p) + \mathrm{KL}(p\|q)$

- **Descomposición:**
  - $H(p)$: incertidumbre "irreductible" de la fuente (no depende de nuestro modelo),
  - $\mathrm{KL}(p\|q)$: **penalización extra** por usar el modelo $q$ en lugar del modelo óptimo $p$.
- **Mensaje:** al minimizar $H(p,q)$ sobre $q$, en realidad estamos minimizando $\mathrm{KL}(p\|q)$, ya que $H(p)$ es constante.
- Esto anticipa lo que haremos en ML: elegir parámetros para que la distribución predicha por el modelo se acerque a la distribución verdadera de etiquetas.

# Setup: clasificación probabilística y etiquetas one-hot

- Problema de clasificación con $C$ clases.
- Dado un input $x$, el modelo produce un vector de probabilidades:

$$\hat{p}_\theta(y = c \mid x), \quad c = 1, \ldots, C.$$

- La etiqueta verdadera se representa como vector **one-hot**:

$$y = (y_1, \ldots, y_C), \quad y_c \in \{0, 1\}, \quad \sum_c y_c = 1.$$

- **Cross-entropy empírica** para un ejemplo $(x, y)$:

$$\ell_{\mathrm{CE}}(y, \hat{p}_\theta(\cdot \mid x)) = -\sum_{c=1}^{C} y_c \log \hat{p}_\theta(y = c \mid x).$$

- Como solo una componente de $y$ es 1, la suma se reduce a $-\log$ de la probabilidad de la clase verdadera.

# De cross-entropy a negative log-likelihood

- Si la clase verdadera es $k$, entonces la etiqueta one-hot cumple:

$$y_k = 1, \qquad y_c = 0 \ \text{ para } c \neq k.$$

- Sustituyendo en la definición:

$$\ell_{\mathrm{CE}}(y, \hat{p}) = -\sum_{c=1}^{C} y_c \log \hat{p}(c) = -\log \hat{p}(k).$$

- **Interpretación:** la loss depende solo de la probabilidad asignada a la clase correcta.
- Por eso en clasificación probabilística, la CE para un ejemplo coincide con la **negative log-likelihood**:

$$\ell_{\mathrm{NLL}} = -\log \hat{p}_\theta(y \mid x).$$

# Ejemplos numéricos: modelo seguro y modelo inseguro

- Supón 3 clases: gato, perro, ave.
- Etiqueta verdadera: $y = (1, 0, 0)$ (gato).
- Caso A (modelo acierta con alta confianza):

$$\hat{p} = (0{,}9, 0{,}05, 0{,}05) \Rightarrow \ell_{\mathrm{CE}} = -\log 0{,}9 \approx 0{,}105.$$

- Caso B (modelo acierta pero poco confiado):

$$\hat{p} = (0{,}5, 0{,}25, 0{,}25) \Rightarrow \ell_{\mathrm{CE}} = -\log 0{,}5 \approx 0{,}693.$$

- Caso C (modelo se equivoca con mucha confianza, cree que es perro):

$$\hat{p} = (0{,}01, 0{,}98, 0{,}01) \Rightarrow \ell_{\mathrm{CE}} = -\log 0{,}01 \approx 4{,}605.$$

- **Lectura:** la cross-entropy **diferencia** fuertemente entre "equivocarse con duda" y "equivocarse con confianza".

# Diferencia entre accuracy y cross-entropy

- **Accuracy:** porcentaje de ejemplos donde $\arg\max_c \hat{p}_\theta(y = c \mid x)$ coincide con la clase verdadera.
- **Problema:** no distingue entre:
  - $\hat{p} = (0{,}51, 0{,}49)$ (muy inseguro),
  - $\hat{p} = (0{,}99, 0{,}01)$ (muy seguro),

  si en ambos casos acierta la clase.
- **Cross-entropy:** premia modelos que aciertan y además asignan alta probabilidad a la clase verdadera.
- En entrenamiento, **optimizamos cross-entropy**; accuracy suele usarse como métrica de evaluación secundaria.

# Derivación de la entropía como esperanza de surprisal

- Definimos información de un evento: $I(x) = -\log p(x)$.
- Para una variable aleatoria $X \sim p$, la información promedio es:

$$\mathbb{E}_p[I(X)] = \sum_x p(x)\, I(x) = \sum_x p(x)\,[-\log p(x)].$$

$$\Rightarrow \quad H(X) = -\sum_x p(x) \log p(x).$$

- **Comentario pedagógico:** enfatizar en clase que **entropía no es una fórmula mágica**, sino simplemente la esperanza de la sorpresa.

# Derivación de CE en clasificación one-hot

- Partimos de la definición general de cross-entropy para una etiqueta $y$ y una predicción $\hat{p}$:

$$\ell_{\mathrm{CE}}(y, \hat{p}) = -\sum_{c=1}^{C} y_c \log \hat{p}(c).$$

- Si la clase correcta es $k$, entonces $y_k = 1$ y todas las demás componentes valen 0.
- Por lo tanto, todos los términos se anulan excepto el de la clase verdadera:

$$\ell_{\mathrm{CE}}(y, \hat{p}) = -(0 + \cdots + 1 \cdot \log \hat{p}(k) + \cdots + 0).$$

$$\Rightarrow \quad \ell_{\mathrm{CE}}(y, \hat{p}) = -\log \hat{p}(k).$$

- Esta es exactamente la forma de la **negative log-likelihood** para una observación etiquetada.

# Ejemplo 1 — entropía de una distribución sobre tres eventos

- $X \in \{rojo,\ azul,\ verde\}$ con:

$$p(rojo) = 0{,}1, \quad p(azul) = 0{,}2, \quad p(verde) = 0{,}7.$$

- Calcula:

$$H(X) = -\big[0{,}1 \log_2 0{,}1 + 0{,}2 \log_2 0{,}2 + 0{,}7 \log_2 0{,}7\big].$$

- Valores aproximados:

$$\log_2 0{,}1 \approx -3{,}322, \quad \log_2 0{,}2 \approx -2{,}322, \quad \log_2 0{,}7 \approx -0{,}515.$$

$$H(X) \approx -\big[0{,}1(-3{,}322) + 0{,}2(-2{,}322) + 0{,}7(-0{,}515)\big]$$
$$\approx -\big[-0{,}3322 - 0{,}4644 - 0{,}3605\big] \approx 1{,}157 \text{ bits}.$$

- **Comparar:** con la entropía de la distribución uniforme en 3 valores ($\approx 1{,}585$ bits): aquí la distribución está más concentrada, por eso la entropía es menor.

# Ejemplo 2 — cross-entropy y KL entre $p$ y $q$

- Sea $\mathcal{X} = \{0, 1\}$.
- Verdadera $p$: $\quad p(1) = 0{,}7,\ p(0) = 0{,}3.$
- Modelo $q$: $\quad q(1) = 0{,}4,\ q(0) = 0{,}6.$
- Entropía de $p$:
$$H(p) = -[0{,}7 \log_2 0{,}7 + 0{,}3 \log_2 0{,}3] \approx 0{,}881 \text{ bits.}$$
- Cross-entropy:
$$H(p, q) = -[0{,}7 \log_2 0{,}4 + 0{,}3 \log_2 0{,}6] \approx 1{,}286 \text{ bits.}$$
- KL:
$$\mathrm{KL}(p \| q) = H(p, q) - H(p) \approx 1{,}286 - 0{,}881 = 0{,}405 \text{ bits.}$$
- **Lectura:** el modelo $q$ paga 0.405 bits extra por símbolo respecto al modelo óptimo que conoce $p$.

# Ejemplo 3 — CE en clasificación binaria

- Clasificación binaria ($y \in \{0, 1\}$).
- Datos:
  - Ejemplo 1: $y = 1$, $\hat{p}(1 \mid x) = 0{,}9$.
  - Ejemplo 2: $y = 1$, $\hat{p}(1 \mid x) = 0{,}6$.
  - Ejemplo 3: $y = 1$, $\hat{p}(1 \mid x) = 0{,}2$.
- Loss individual:

$$\ell_1 = -\log 0{,}9 \approx 0{,}105, \quad \ell_2 = -\log 0{,}6 \approx 0{,}511, \quad \ell_3 = -\log 0{,}2 \approx 1{,}609.$$

- Loss promedio:

$$\bar{\ell} = \frac{\ell_1 + \ell_2 + \ell_3}{3} \approx \frac{0{,}105 + 0{,}511 + 1{,}609}{3} \approx 0{,}742.$$

- **Discusión para clase:** ¿qué cambio en el modelo bajaría más la loss? (hacer que el modelo deje de estar tan seguro en el ejemplo 3).

# Entropía como medida de incertidumbre del modelo

- Dado un vector de probabilidades de clase $\hat{p}_\theta(\cdot \mid x)$, podemos calcular:

$$H(\hat{p}_\theta(\cdot \mid x)) = -\sum_c \hat{p}_\theta(y = c \mid x) \log \hat{p}_\theta(y = c \mid x).$$

- **Alta entropía:** el modelo está indeciso (probabilidades dispersas).
- **Baja entropía:** el modelo está muy seguro (probabilidades concentradas).
- En active learning o detección de incertidumbre, se pueden priorizar ejemplos con entropía alta.

# Cross-entropy como loss natural en clasificación

- Si el dataset es una muestra iid de la distribución verdadera, la CE empírica aproxima:

$$\mathbb{E}_{(x,y)\sim p_{\text{data}}}\big[-\log \hat{p}_\theta(y \mid x)\big] \approx H(p_{\text{data}}, \hat{p}_\theta).$$

- Minimizar la CE empírica $\Rightarrow$ acercar $\hat{p}_\theta$ a $p_{\text{data}}$ en el sentido de $\mathrm{KL}(p_{\text{data}} \| \hat{p}_\theta)$.
- **Conexión con MLE:** MLE elige $\theta$ que maximiza la probabilidad del dataset; es equivalente a minimizar la NLL o la CE promedio.
- En la práctica, esto se implementa como:

$$\min_\theta \frac{1}{N} \sum_{i=1}^{N} -\log \hat{p}_\theta(y_i \mid x_i).$$

# ¿Por qué aparece KL en métodos probabilísticos avanzados?

- En **variational inference**, aproximamos una distribución posterior $p(z \mid x)$ con una familia $q_\phi(z)$.
- Elegimos $\phi$ para que $q_\phi$ esté "cerca" de $p$, típicamente minimizando:

$$\mathrm{KL}(q_\phi(z) \| p(z \mid x)).$$

- En **VAEs** y otros modelos generativos, la función de entrenamiento incluye un término de KL que regulariza la distribución latente hacia un prior simple (ej. $\mathcal{N}(0, I)$).
- En **model distillation**, se usa KL para alinear las distribuciones de salida de un "teacher" y un "student".
- **Idea transversal:** KL es la *moneda estándar* para cuantificar "qué tan diferente" es un modelo probabilístico de otro.

# Errores típicos de interpretación

- **Error 1:** pensar que entropía mide "desorden físico" en este contexto.
  **Corrección:** aquí es una medida de **incertidumbre** de una variable aleatoria.
- **Error 2:** creer que cross-entropy solo importa "porque funciona bien en práctica".
  **Corrección:** está directamente conectada con MLE y con la idea de codificación óptima.
- **Error 3:** interpretar KL como una distancia "simétrica".
  **Corrección:** KL no es simétrica y el orden de $p$ y $q$ importa mucho.
- **Error 4:** pensar que alta accuracy $\Rightarrow$ cross-entropy baja.
  **Corrección:** se puede tener buena accuracy pero probabilidades muy mal calibradas.

# Ideas clave (Semana 6)

- **Self-information:** $I(x) = -\log p(x)$ mide cuán inesperado fue un evento.
- **Entropía:** $H(X) = \mathbb{E}[I(X)]$ es incertidumbre promedio; máxima para distribuciones uniformes.
- **Cross-entropy:** $H(p, q) = \mathbb{E}_p[-\log q]$ es el costo de codificar datos de $p$ usando el modelo $q$.
- **KL divergence:** $\mathrm{KL}(p\|q) = H(p, q) - H(p)$ mide el "precio extra" por usar el modelo incorrecto.
- **Identidad clave:** $H(p, q) = H(p) + \mathrm{KL}(p\|q)$.
- **En ML:** minimizar cross-entropy $\Leftrightarrow$ hacer MLE $\Leftrightarrow$ aproximar la distribución verdadera de etiquetas.
- **Etiquetas one-hot:** la loss se reduce a $-\log$ de la probabilidad de la clase correcta.

# Mapa de fórmulas clave (Information Theory)

| **Fórmula** | **Interpretación** |
|---|---|
| $I(x) = -\log p(x)$ | Sorpresa/información de un evento. |
| $H(X) = -\sum_x p(x) \log p(x)$ | Incertidumbre promedio de $X$. |
| $H(p, q) = -\sum_x p(x) \log q(x)$ | Costo de codificar datos de $p$ con modelo $q$. |
| $\mathrm{KL}(p\|q) = \sum_x p(x) \log \frac{p(x)}{q(x)}$ | Penalización por usar $q$ en lugar de $p$. |
| $H(p, q) = H(p) + \mathrm{KL}(p\|q)$ | Descomposición de cross-entropy en incertidumbre + error de modelo. |
| $\mathrm{NLL}(\theta) = -\frac{1}{N} \sum_i \log \hat{p}_\theta(y_i \mid x_i)$ | Loss promedio en MLE; coincide con CE promedio. |
| $\ell_{\mathrm{CE}}(y, \hat{p}) = -\sum_c y_c \log \hat{p}(c)$ | Cross-entropy entre etiqueta one-hot y distribución predicha. |

# Semana 6 — mapa del temario (progreso)

- **Teoría**
- **Lab NumPy**
- **Lab sklearn**
- **Entregable**

# Lab NumPy — objetivo general

- Implementar $H(p) = -\sum_x p(x) \log p(x)$, $H(p,q)$ y $\mathrm{KL}(p\|q)$ en ejemplos discretos pequeños (tablas $2 \times 1$, $3 \times 1$).
- Graficar $H(\theta) = -\theta \log_2 \theta - (1-\theta) \log_2 (1-\theta)$ para $\theta \in (0,1)$ y contrastar con la figura de la sesión.

# Lab NumPy — checklist (qué debe aparecer en tu notebook)

- **Funciones:** entropía de un vector de probabilidades (con chequeo $\sum p_i = 1$); cross-entropy y KL entre dos vectores alineados.
- **Casos borde:** evitar $\log 0$ (clamp o máscara); verificar $\mathrm{KL}(p\|p) = 0$ numéricamente.
- **Bernoulli:** vector $\theta$ en $(0, 1)$ y gráfico de $H(\theta)$; marcar $\theta = 0{,}5$.

# Semana 6 — mapa del temario (progreso)

- **Teoría**
- **Lab NumPy**
- **Lab sklearn**
- **Entregable**

# Lab sklearn — objetivo general

- Usar `sklearn.metrics.log_loss` como CE promedio (base $e$ por defecto; documentar la base).
- Comparar con tu implementación NumPy en el mismo conjunto de etiquetas y probabilidades predichas.

# Lab sklearn — checklist (estructura sugerida)

- **Datos:** un problema de clasificación pequeño (binario o multiclase) con `predict_proba`.
- **Métrica:** `log_loss(y_true, y_proba)` y desglose manual en $-\frac{1}{N}\sum_i \log \hat{p}(y_i \mid x_i)$.
- **Discusión:** por qué accuracy puede ocultar mala calibración; relacionar con slides de CE vs accuracy.

# Semana 6 — mapa del temario (progreso)

- **Teoría**
- **Lab NumPy**
- **Lab sklearn**
- **Entregable**

# Entregable — qué se evalúa

- **Slides:** definiciones claras de surprisal, entropía, CE y KL; identidad $H(p,q) = H(p) + \mathrm{KL}(p\|q)$; vínculo CE $\leftrightarrow$ NLL/MLE.
- **Notebook NumPy:** funciones de entropía / CE / KL reproducibles y gráfica de entropía Bernoulli.
- **Notebook sklearn:** `log_loss` contrastado con cálculo manual.

# Cierre de la Semana 6

- Information theory proporciona el **lenguaje** para hablar de incertidumbre y de modelos probabilísticos.
- Las pérdidas estándar en clasificación (cross-entropy / NLL) no son "trucos": vienen directamente de estas ideas.
- La KL divergence reaparecerá en casi todos los métodos probabilísticos avanzados que veamos después.

# Semana 7 — Álgebra lineal para Machine Learning

## Proyección, mínimos cuadrados, SVD, conditioning y PCA

Curso de Machine Learning — UAQ (Matemáticas Aplicadas)

Libro base: *Probabilistic Machine Learning: An Introduction* (Kevin P. Murphy)

Semana 7

# Lecturas sugeridas y ruta matemática

- **Cap. 7 — Linear Algebra**
  - **7.1.2–7.1.4** Espacios vectoriales, normas, propiedades matriciales
  - **7.4** EVD de matrices simétricas y formas cuadráticas
  - **7.5** SVD, conexión con EVD, truncación y pseudoinversa
  - **7.6.2** QR
  - **7.7.3** Sistemas sobredeterminados y mínimos cuadrados
- **Conexiones prácticas**
  - **11.2** Regresión lineal por mínimos cuadrados
  - **20.1** PCA y su derivación espectral

**Ruta de la semana**

**modelo lineal $\to$ proyección $\to$ mínimos cuadrados $\to$ conditioning y resolución estable $\to$ SVD $\to$ PCA**

# Pregunta central de la semana

Dado un vector $y \in \mathbb{R}^n$ y una matriz $X$, queremos responder cuatro preguntas:

1. ¿Cuál es la mejor aproximación de $y$ dentro de $\mathrm{col}(X)$?
2. ¿Qué ecuación caracteriza esa solución?
3. ¿Cómo resolverla de forma estable cuando el problema está mal condicionado?
4. ¿Qué nos revela la SVD sobre rango, energía, truncación, pseudoinversa y PCA?

**Mensaje**

**La geometría del modelo lineal y la estructura espectral de las matrices responden a la misma pregunta: cómo aproximar y representar información en subespacios.**

# Semana 7 — mapa del temario (progreso)

- **Teoría**
- **Lab NumPy**
- **Lab sklearn**
- **Entregable**

# El modelo lineal como subespacio

- Si $X = [x_1, \ldots, x_p] \in \mathbb{R}^{n \times p}$, cualquier combinación lineal de sus columnas tiene la forma

$$Xw = w_1 x_1 + \cdots + w_p x_p.$$

- El conjunto de todas esas combinaciones es el **espacio columna**

$$\mathrm{col}(X) = \{Xw : \ w \in \mathbb{R}^p\}.$$

- Un modelo lineal no puede producir cualquier vector en $\mathbb{R}^n$: solo vectores dentro de $\mathrm{col}(X)$.

**Idea central**

**Regresión lineal = buscar el mejor vector posible dentro de $\mathrm{col}(X)$.**

# Norma euclidiana y error cuadrático

- La norma euclidiana de un vector es
$$\|x\|_2 = \sqrt{x^\top x}.$$
- La distancia entre $y$ y una predicción $Xw$ es
$$\|y - Xw\|_2.$$
- El problema de mínimos cuadrados se formula como
$$\hat{w} = \arg\min_w \|y - Xw\|_2^2.$$

**Lectura geométrica**

**Mínimos cuadrados es un problema de distancia mínima entre un vector y un subespacio.**

# Proyección ortogonal sobre una recta

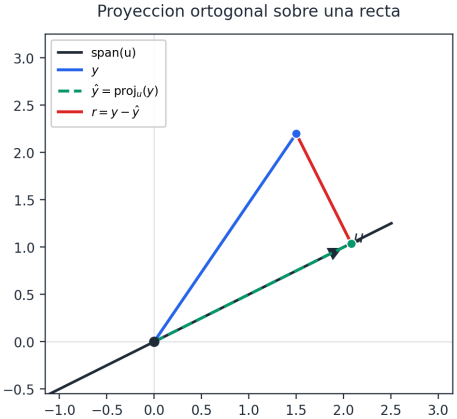
Queremos proyectar $y$ sobre la dirección de $u$. Buscamos $a$ tal que $au$ minimice

$$\|y - au\|_2^2.$$

$$f(a) = (y - au)^\top (y - au) = y^\top y - 2a\, u^\top y + a^2 u^\top u,$$
$$f'(a) = -2u^\top y + 2a\, u^\top u = 0.$$

Por tanto,

$$a = \frac{u^\top y}{u^\top u} \Rightarrow \mathrm{proj}_u(y) = \frac{u^\top y}{u^\top u}\, u.$$



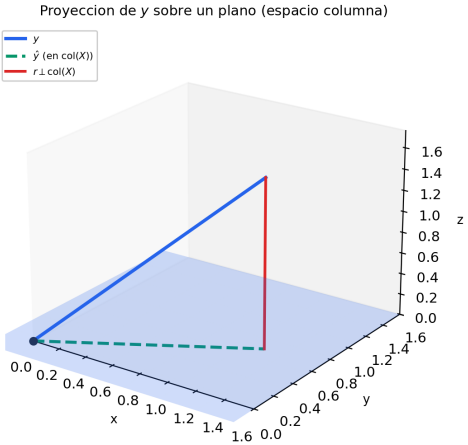

- El residual $r = y - \hat{y}$ satisface $u^\top r = 0$: el error es ortogonal a la dirección usada para aproximar.

# Mínimos cuadrados como proyección sobre $\mathrm{col}(X)$

$$\hat{w} = \arg\min_{w} \|y - Xw\|_2^2, \quad \hat{y} = X\hat{w}.$$

- $\hat{y}$ es la **proyección ortogonal** de $y$ sobre $\mathrm{col}(X)$.
- El residual $r = y - \hat{y}$ es ortogonal a todas las columnas de $X$.
- Por tanto, $X^\top r = 0$.

Esquema en $\mathbb{R}^3$: el plano representa $\mathrm{col}(X)$. En la práctica, el razonamiento vale en $\mathbb{R}^n$.



# Ecuaciones normales

Como $r = y - X\hat{w}$ y $X^\top r = 0$,

$$X^\top(y - X\hat{w}) = 0 \qquad \Rightarrow \qquad X^\top X\hat{w} = X^\top y.$$

**Resultado**

**Las ecuaciones normales caracterizan algebraicamente la proyección ortogonal de $y$ sobre $\mathrm{col}(X)$.**

También se obtiene por gradiente:

$$J(w) = \|y - Xw\|_2^2 \quad \Rightarrow \quad \nabla_w J(w) = -2X^\top y + 2X^\top Xw = 0.$$

# Matriz de proyección

Si $X^\top X$ es invertible, entonces

$$\hat{w} = (X^\top X)^{-1} X^\top y, \qquad \hat{y} = X(X^\top X)^{-1} X^\top y.$$

Definimos

$$P = X(X^\top X)^{-1} X^\top.$$

Entonces

$$\hat{y} = Py.$$

- $P^\top = P$ (simétrica).
- $P^2 = P$ (idempotente).

**Interpretación**

**$P$ es la matriz que proyecta ortogonalmente sobre $\mathrm{col}(X)$.**

# Ejemplo completo de mínimos cuadrados

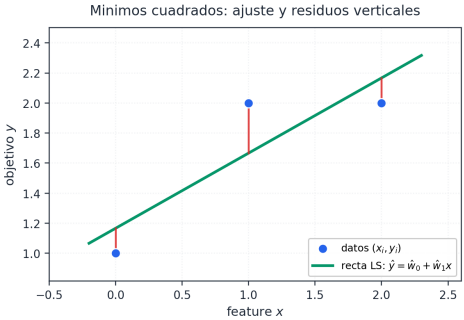
$$X = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}, \quad y = \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix}.$$

$$\hat{w} = \begin{bmatrix} 7/6 \\ 1/2 \end{bmatrix}, \ \hat{y} = \begin{bmatrix} 7/6 \\ 10/6 \\ 13/6 \end{bmatrix}, \ r = y - \hat{y} = \frac{1}{6}\begin{bmatrix} -1 \\ 2 \\ -1 \end{bmatrix}.$$

Se verifica que $X^\top r = 0$.

## Lectura

**La recta de regresión es la proyección ortogonal del vector de respuestas sobre el subespacio generado por las columnas de $X$.**



# Conditioning: sensibilidad de la solución

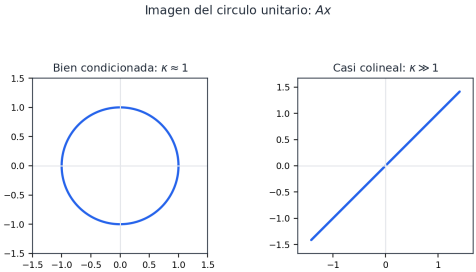
Un problema lineal está **mal condicionado** si pequeñas perturbaciones en la entrada producen grandes cambios en la solución.

$$\kappa(A) = \frac{\sigma_{\text{máx}}(A)}{\sigma_{\text{mín}}(A)}$$

donde, en la norma 2:

- $\sigma_{\text{máx}}(A)$ es el **mayor valor singular** de $A$;
- $\sigma_{\text{mín}}(A)$ es el **menor valor singular no nulo** de $A$.
- Los $\sigma_i$ miden cuánto **estira** o **comprime** la matriz en direcciones especiales.
- Si $\sigma_{\text{mín}}(A)$ es muy pequeño, una dirección casi se colapsa y el error se amplifica.

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 1{,}0001 \end{bmatrix}$$



# Cómo resolver mínimos cuadrados de forma estable

Queremos resolver siempre la misma pregunta:

$$\min_{w} \|y - Xw\|_2^2.$$

1. **Ecuaciones normales**
$$X^\top X w = X^\top y$$
excelentes para la intuición geométrica, pero pueden ser numéricamente delicadas.
2. **QR**
$$X = QR$$
evita formar $X^\top X$ y suele ser más estable.
3. **SVD**
$$X = U \Sigma V^\top$$
funciona incluso con rango deficiente y conduce naturalmente a la pseudoinversa.

## Mensaje

**No son tres problemas distintos: son tres maneras de responder la misma pregunta de mínimos cuadrados.**

# La descomposición singular de una matriz

## Definición

Para toda matriz real $A \in \mathbb{R}^{m \times n}$ existe una factorización

$$A = U \Sigma V^{\top},$$

donde $U \in \mathbb{R}^{m \times m}$ y $V \in \mathbb{R}^{n \times n}$ son ortogonales, y $\Sigma \in \mathbb{R}^{m \times n}$ es diagonal rectangular con

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq 0.$$

- Las columnas de $U$ son los **vectores singulares izquierdos**.
- Las columnas de $V$ son los **vectores singulares derechos**.
- Los $\sigma_i$ son los **valores singulares**.

## Observación

**La SVD siempre existe; la EVD no siempre existe, incluso para matrices cuadradas.**

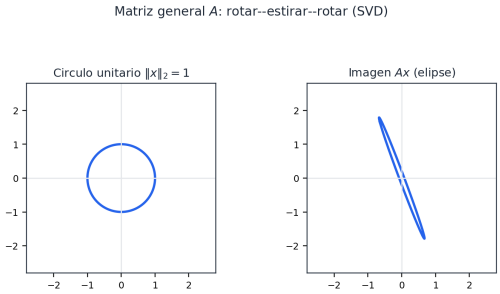
# Interpretación geométrica de la SVD

La descomposición $A = U\Sigma V^\top$ se interpreta como tres pasos:

1. $V^\top$: cambia de base en el espacio de entrada;
2. $\Sigma$: estira o comprime en direcciones ortogonales especiales;
3. $U$: rota o refleja en el espacio de salida.

**Conclusión:**

**Toda matriz actúa como: rotar/reflejar $\to$ escalar $\to$ rotar/reflejar.**



# Conexión entre SVD y EVD

Si

$$A = U\Sigma V^\top,$$

entonces

$$A^\top A = V\Sigma^\top \Sigma V^\top, \qquad AA^\top = U\Sigma\Sigma^\top U^\top.$$

- Las columnas de $V$ son eigenvectores de $A^\top A$.
- Las columnas de $U$ son eigenvectores de $AA^\top$.
- Los eigenvalores no nulos son

$$\lambda_i = \sigma_i^2.$$

## Caso especial

Si $A$ es simétrica y definida positiva, entonces la SVD coincide con la EVD:

$$A = U\Sigma U^\top.$$

# ¿Por qué las columnas de $V$ son eigenvectores de $A^\top A$?

Partimos de la SVD $A = U\Sigma V^\top$. Entonces

$$A^\top A = V\Sigma^\top U^\top U\Sigma V^\top = V\Sigma^\top \Sigma V^\top.$$

Sea $v_i$ la columna $i$-ésima de $V$. Como $V$ es ortogonal, $V^\top v_i = e_i$ (vector canónico). Por tanto

$$A^\top A\, v_i = V\Sigma^\top \Sigma e_i = V(\sigma_i^2 e_i) = \sigma_i^2 v_i.$$

**Conclusión:** $v_i$ es eigenvector de $A^\top A$ con eigenvalor $\sigma_i^2$.
**Interpretación:**

- $A^\top A$ actúa en el espacio de entrada.
- Las direcciones especiales del espacio de entrada están dadas por las columnas de $V$.
- Por eso $V$ aparece en la descomposición espectral de $A^\top A$.

## Mensaje final

Las columnas de $V$ diagonalizan a $A^\top A$, y sus eigenvalores son los cuadrados de los valores singulares.

# Comprobación numérica: $\sigma_i^2 = \lambda_i(A^\top A)$

Sea

$$A = \begin{bmatrix} 1 & 2 \\ 0 & 2 \\ 2 & 0 \end{bmatrix}.$$

Entonces

$$A^\top A = \begin{bmatrix} 5 & 2 \\ 2 & 8 \end{bmatrix}.$$

Sus eigenvalores son

$$\lambda_1 = 9, \qquad \lambda_2 = 4.$$

Luego, los valores singulares de $A$ son

$$\sigma_1 = 3, \qquad \sigma_2 = 2.$$

Por tanto,

$$\sigma_1^2 = 9 = \lambda_1(A^\top A), \qquad \sigma_2^2 = 4 = \lambda_2(A^\top A).$$

## Mensaje

Los valores singulares de $A$ son la raíz cuadrada de los eigenvalores de $A^\top A$.

# Energía, rango y truncación

La SVD puede escribirse como suma de matrices de rango 1:

$$A = \sum_{i=1}^{r} \sigma_i u_i v_i^\top,$$

donde $r = \mathrm{rank}(A)$.

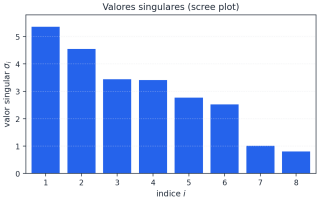
- La energía total en norma de Frobenius es

$$\|A\|_F^2 = \sum_{i=1}^{r} \sigma_i^2.$$

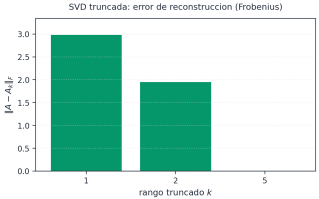
- La aproximación truncada de rango $k$ es

$$A_k = \sum_{i=1}^{k} \sigma_i u_i v_i^\top.$$

- Los primeros valores singulares concentran la estructura principal de la matriz.




Scree plot.




Error de reconstrucción.

# Ejemplo: compresión de imágenes con SVD

Una imagen en escala de grises puede representarse como una matriz

$$X \in R^{m \times n}.$$

Aplicamos la descomposición singular:

$$X = U \Sigma V^T.$$

Si conservamos solo los primeros $k$ valores singulares, obtenemos la aproximación truncada

$$X_k = U_{:,1:k} \Sigma_{1:k,1:k} V^T_{:,1:k}.$$

Ideas clave:

- $X_k$ tiene rango $k$.
- Los primeros valores singulares capturan la estructura dominante de la imagen.
- Si la imagen tiene redundancia, puede aproximarse bien con pocos componentes.
- Esto muestra que la SVD sirve para compresión y aproximación de bajo rango.

## Mensaje final

La SVD permite describir una imagen compleja usando solo unas cuantas direcciones principales.

# Espectro singular y estructura de una imagen

Si graficamos los valores singulares $\sigma_1, \sigma_2, \ldots$, observamos cuánto aporta cada componente de la SVD.

**Qué observar:**

- En una imagen real, los valores singulares suelen decrecer relativamente rápido.
- Esto indica que gran parte de la información está concentrada en pocos modos.
- Si aleatorizamos los pixeles, la estructura geométrica se destruye.
- En ese caso, el espectro singular se vuelve menos concentrado.

**Interpretación:**

- El decaimiento de los valores singulares mide la complejidad efectiva de la matriz.

## Mensaje final

Una matriz con estructura puede comprimirse mejor que una matriz aleatoria.

# Pseudo-inversa de Moore–Penrose

La pseudo-inversa de Moore–Penrose de $A$, denotada por $A^\dagger$, es la **única** matriz que satisface:

$$AA^\dagger A = A, \qquad A^\dagger AA^\dagger = A^\dagger,$$

$$(AA^\dagger)^\top = AA^\dagger, \qquad (A^\dagger A)^\top = A^\dagger A.$$

- Si $A$ es cuadrada e invertible, entonces $A^\dagger = A^{-1}$.
- Si $A$ es **tall** $(m > n)$ y full rank en columnas,

$$A^\dagger = (A^\top A)^{-1} A^\top.$$

- Si $A$ es **fat** $(m < n)$ y full rank en filas,

$$A^\dagger = A^\top (AA^\top)^{-1}.$$

# Pseudoinversa vía SVD y lectura geométrica

Si

$$A = U\Sigma V^\top,$$

entonces

$$A^\dagger = V\Sigma^\dagger U^\top,$$

donde $\Sigma^\dagger$ invierte los valores singulares no nulos y deja en cero los demás.

- En problemas **sobredeterminados**, $A^\dagger b$ da la solución de mínimos cuadrados.
- En problemas **infradeterminados**, $A^\dagger b$ da la solución exacta de **menor norma** cuando existe.
- La pseudoinversa unifica inversa, mínimos cuadrados y rango deficiente en una sola fórmula.

## Ejemplo que ya vimos

Para $A = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}$ y $b = \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix}$,

$$A^\dagger b = \begin{bmatrix} 7/6 \\ 1/2 \end{bmatrix},$$

que coincide con el estimador OLS.

# Formas cuadráticas y cociente de Rayleigh

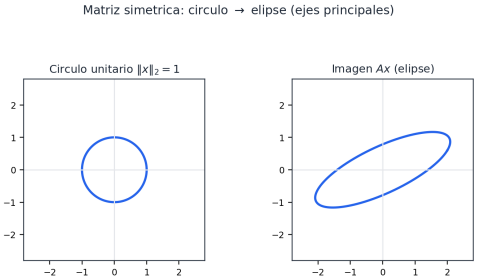
Sea $A$ una matriz simétrica. La forma cuadrática asociada es

$$q(v) = v^\top A v.$$

El **cociente de Rayleigh** de $A$ en una dirección $v \neq 0$ es

$$R_A(v) = \frac{v^\top A v}{v^\top v}.$$

Si además $\|v\| = 1$, entonces $R_A(v) = v^\top A v$.




## Resultado fundamental

Si $A$ es simétrica, el máximo de $R_A(v)$ se alcanza en un eigenvector del mayor eigenvalor, y el mínimo en un eigenvector del menor eigenvalor.

# Ejemplo simétrico $2 \times 2$

Sea

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}.$$

Entonces

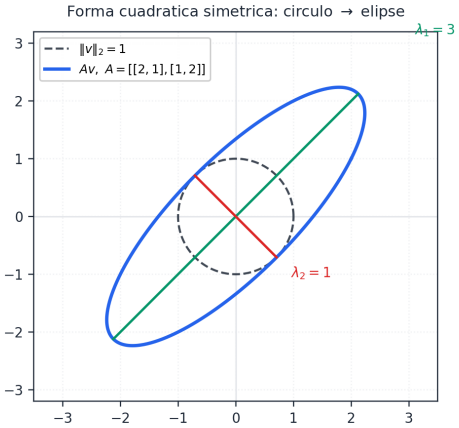
$$q(v) = v^\top A v = 2v_1^2 + 2v_2^2 + 2v_1 v_2.$$

Sus eigenpares son:

$$\lambda_1 = 3, \quad q_1 = \tfrac{1}{\sqrt{2}}(1,1)^\top, \quad \lambda_2 = 1, \quad q_2 = \tfrac{1}{\sqrt{2}}(1,-1)^\top.$$

Sobre la esfera unitaria,

$$\text{máx}\, q(v) = 3 \text{ en } q_1, \quad \text{mín}\, q(v) = 1 \text{ en } q_2.$$



# PCA como problema espectral

Sea $X_c$ la matriz de datos centrada. PCA busca la dirección unitaria que maximiza la varianza proyectada:

$$\max_{\|v\|=1} v^\top X_c^\top X_c v.$$

Con multiplicadores de Lagrange,

$$\mathcal{L}(v, \lambda) = v^\top X_c^\top X_c v - \lambda(v^\top v - 1)$$

implica

$$X_c^\top X_c v = \lambda v.$$

- Las componentes principales son eigenvectores de $X_c^\top X_c$.
- El mayor eigenvalor corresponde a la primera componente principal.

# PCA y SVD de datos centrados

Si

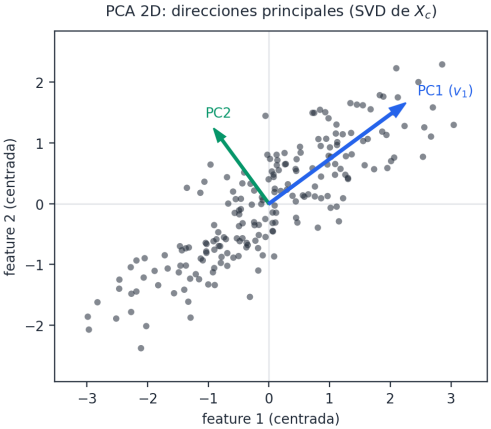
$$X_c = U\Sigma V^\top,$$

entonces

$$X_c^\top X_c = V\Sigma^\top \Sigma V^\top.$$

Por tanto:

- las columnas de $V$ son las **direcciones principales**;
- la varianza explicada está asociada a $\sigma_i^2$ (salvo factores de escala como $n$ o $n-1$);
- PCA es la aplicación estadística más importante de la estructura espectral vista en esta semana.



# Cierre de la semana

- **Proyección, mínimos cuadrados y regresión lineal** describen el mismo objeto geométrico.
- **Conditioning** explica cuándo una solución es sensible al ruido y por qué conviene resolver con QR o SVD.
- **SVD** organiza la matriz en modos ortogonales, cuantifica energía, permite truncar y define la pseudoinversa.
- **Formas cuadráticas, cociente de Rayleigh y EVD** preparan el camino hacia **PCA**.

*Detrás de muchos algoritmos de machine learning hay una misma idea: representar, aproximar y descomponer información en espacios lineales.*

# Semana 7 — mapa del temario (progreso)

- **Teoría**
- **Lab NumPy**
- **Lab sklearn**
- **Entregable**

# Lab NumPy — Actividad 1: proyección en una dirección

- Fijar $u, y \in \mathbb{R}^d$; calcular
$$\hat{y} = \frac{u^\top y}{u^\top u} u, \qquad r = y - \hat{y}.$$
- Verificar numéricamente que $u^\top r \approx 0$.

# Lab NumPy — Actividad 2: mínimos cuadrados del ejemplo de clase

- Construir $X$ y $y$ del ejemplo; calcular $X^\top X$, $X^\top y$, $\hat{w}$, $\hat{y}$ y $r$.
- Verificar $X^\top r \approx 0$.
- Comparar con `numpy.linalg.lstsq`.

# Lab NumPy — Actividades 3–4: SVD y conditioning

- **SVD:** usar `np.linalg.svd`, reconstruir $A$ y aproximarla por $A_k$; medir error Frobenius.
- **Conditioning:** comparar matrices bien y mal condicionadas con `np.linalg.cond`; perturbar $b$ en $Ax = b$ y observar el efecto sobre la solución.

# Lab NumPy — Pseudocódigo: compresión de imágenes con SVD

**Pseudocódigo (guía conceptual)**

1. Cargar una imagen en escala de grises y representarla como una matriz $X$.
2. Calcular la SVD:
$$X = U\Sigma V^T.$$
3. Elegir varios valores de $k$, por ejemplo:
$$k \in \{1, 2, 5, 10, 20, 50, 100\}.$$
4. Para cada $k$, construir la aproximación truncada:
$$X_k = U_{:,1:k}\Sigma_{1:k,1:k}V^T_{:,1:k}.$$
5. Visualizar cada reconstrucción y comparar la pérdida de detalle.
6. Graficar los primeros valores singulares para observar su decaimiento.
7. Aleatorizar los pixeles de la imagen, recalcular la SVD y comparar el espectro singular con el de la imagen original.
8. Concluir qué relación hay entre:
   - estructura de la matriz,
   - rapidez del decaimiento espectral,
   - calidad de la compresión.

# Preguntas para discutir en laboratorio

- ¿Por qué una imagen real puede comprimirse bien con pocos componentes?
- ¿Qué cambia cuando se destruye la estructura espacial de los pixeles?
- ¿Qué información nos da el espectro singular sobre la complejidad de la matriz?
- ¿Cómo conecta este experimento con PCA y reducción de dimensión?

## Interpretación matemática

El experimento conecta la aproximación de bajo rango

$$X_k = \sum_{i=1}^{k} \sigma_i u_i v_i^\top$$

con la concentración espectral: cuando $\sigma_i$ decae rápido, la energía

$$\|X\|_F^2 = \sum_i \sigma_i^2$$

se preserva con pocos términos y la compresión mantiene estructura relevante.

# Semana 7 — mapa del temario (progreso)

- **Teoría**
- **Lab NumPy**
- **Lab sklearn**
- **Entregable**

# Lab sklearn — Regresión lineal y PCA

- `LinearRegression`: comparar coeficientes con la solución cerrada de mínimos cuadrados.
- `PCA`: interpretar `components_` y `explained_variance_ratio_` a la luz de la SVD de $X_c$.

# Preguntas guía para la discusión

- ¿Qué significa proyectar ortogonalmente?
- ¿Por qué el residual en LS es ortogonal a $\mathrm{col}(X)$?
- ¿Qué indica un valor singular grande frente a uno casi nulo?
- ¿Qué nos dice un número de condición grande sobre la estabilidad del problema?
- ¿Por qué PCA se conecta con eigenvectores de $X_c^\top X_c$ y con la SVD de $X_c$?